铁路信号设备数字孪生系统源代码整理稿

1. 项目概述

铁路信号设备数字孪生系统 是面向铁路信号监测与可视化展示场景的软件系统，采用 Vue 3 + Vite 构建单页应用，结合 Cesium、Three.js 与大屏可视化技术，实现山区铁路信号设备地理展示、数字孪生、恶劣天气演练、告警联动、遥测接入和综合分析展示。

2. 源代码整理说明

- 本整理稿仅纳入当前仓库中真实存在且与系统直接相关的自研代码、配置与辅助服务文件。
- 未纳入第三方依赖、构建产物、静态音视频资源及无关文件。
- 本稿共纳入 20 个文件，总计 14361 行，其中 `src` 目录代码量为 14098 行。
- 代码按基础层、数据与服务层、表现层、参考与辅助层分组，以便后续导出 PDF 与人工核对。

3. 目录结构说明

```
railway_sign/
├── index.html
├── package.json
├── vite.config.js
├── server/
│   └── telemetry-bridge.js
├── src/
│   ├── main.js
│   ├── App.vue
│   ├── config/
│   │   └── demoScenario.js
│   ├── composables/
│   │   └── useApiData.js
│   ├── services/
│   │   ├── api.js
│   │   ├── mockDataService.js
│   │   ├── simulationState.js
│   │   └── telemetrySocket.js
│   ├── components/
│   │   ├── CesiumView.vue
│   │   ├── ThreeView.vue
│   │   ├── DataPanel.vue
│   │   └── datapanel/
│   │       ├── LeftPanel.vue
│   │       ├── CenterPanel.vue
│   │       └── RightPanel.vue
│   └── js/
│       ├── cesium.js
│       └── threejs.js
└── docs/
    └── 本次整理输出材料
```

4. 模块划分说明

- 基础层：应用入口、顶层界面组织、场景参数与工程构建配置。
- 数据与服务层：模拟数据接口、趋势生成、全局状态同步、遥测连接与组合式调用。
- 表现层：Cesium 地图、Three.js 数字孪生与大屏可视化组件。
- 参考与辅助层：早期脚本版本及 Node.js 遥测桥接服务。

3. 模块：基础层

基础层包含应用入口、顶层布局、构建配置与演示场景参数，负责前端应用装配、界面入口组织以及全局基础配置。

3.1 文件：index.html

- 文件说明：前端单页应用挂载点与页面基础容器。
- 行数统计：14 行

```
<!DOCTYPE html>
<html lang="zh-CN">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <link rel="icon" type="image/svg+xml" href="/favicon.svg">
```

```
  <title>山区铁道信号灯数字孪生系统</title>
</head>
<body>
  <div id="app"></div>
  <script type="module" src="/src/main.js"><\/script>
</body>
</html>
```

3.2 文件：vite.config.js

- 文件说明：Vite 构建配置，集成 Vue 与 Cesium 插件。
- 行数统计：18 行

```
import { defineConfig } from 'vite'
import cesium from 'vite-plugin-cesium'
import vue from '@vitejs/plugin-vue'

export default defineConfig({
  plugins: [vue(), cesium()],
  server: {
    port: 4002,
    open: true,
  },
  preview: {
    host: "0.0.0.0",
    port: 4002,
  },
  build: {
    target: "esnext",
  },
});
```

3.3 文件：src/main.js

- 文件说明：Vue 应用入口，挂载根组件。
- 行数统计：5 行

```
import { createApp } from 'vue'
import App from './App.vue'

createApp(App).mount('#app')
```

3.4 文件：src/App.vue

- 文件说明：应用顶层布局与三类主视图切换入口。
- 行数统计：112 行

```
<template>
  <div id="app">
    <div class="tab-container">
      <button
        :class="['tab-btn', { active: currentTab === 'cesium' }]"
        @click="switchTab('cesium')"
      >
        🗺️ 地理信息可视化
      </button>
      <button
        :class="['tab-btn', { active: currentTab === 'three' }]"
        @click="switchTab('three')"
      >
        🚦 铁路信号设备孪生面板
      </button>
      <button
        :class="['tab-btn', { active: currentTab === 'data' }]"
        @click="switchTab('data')"
      >
        📊 数据可视化平台
      </button>
    </div>
```

```
    <div class="content-container">
      <CesiumView v-if="currentTab === 'cesium'" />
      <ThreeView v-else-if="currentTab === 'three'" />
      <DataPanel v-else-if="currentTab === 'data'" />
    </div>
  </div>
</template>

<script setup>
import { ref } from 'vue'
import CesiumView from './components/CesiumView.vue'
import ThreeView from './components/ThreeView.vue'
import DataPanel from './components/DataPanel.vue'

const currentTab = ref('three')

const switchTab = (tab) => {
  currentTab.value = tab
}
<\/script>

<style>
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

html, body, #app {
  width: 100%;
  height: 100%;
  overflow: hidden;
  font-family: 'Microsoft YaHei', Arial, sans-serif;
  background: #000;
}

.tab-container {
  position: fixed;
  top: 0;
  left: 0;
  right: 0;
  height: 50px;
  background: rgba(0, 20, 40, 0.95);
  border-bottom: 2px solid rgba(0, 200, 255, 0.3);
  display: flex;
  justify-content: center;
  align-items: center;
  gap: 20px;
  z-index: 1000;
  backdrop-filter: blur(10px);
  box-shadow: 0 4px 20px rgba(0, 200, 255, 0.2);
}

.tab-btn {
  padding: 8px 20px;
  background: rgba(0, 100, 150, 0.3);
  border: 2px solid rgba(0, 200, 255, 0.3);
  border-radius: 8px;
  color: #aaa;
  font-size: 15px;
  font-weight: bold;
  cursor: pointer;
  transition: all 0.3s;
```

```
  display: flex;
  align-items: center;
  gap: 8px;
}

.tab-btn:hover {
  background: rgba(0, 100, 150, 0.5);
  color: #00d4ff;
  transform: translateY(-2px);
  box-shadow: 0 4px 15px rgba(0, 200, 255, 0.3);
}

.tab-btn.active {
  background: linear-gradient(135deg, #0066cc, #00d4ff);
  color: #fff;
  border-color: #00d4ff;
  box-shadow: 0 0 20px rgba(0, 200, 255, 0.5);
}

.content-container {
  width: 100%;
  height: 100%;
  padding-top: 50px;
}
</style>
```

3.5 文件：src/config/demoScenario.js

- 文件说明：演示场景名称、站点、告警文本等基础业务常量。
- 行数统计：23 行

```
export const DEMO_SCENARIO = {
  regionName: '铁路信号设备数字孪生系统',
  intervalName: '1491-1497 区间演示',
  stationStart: '1491',
  stationEnd: '1497',
  railwayName: '铁路信号设备数字孪生系统',
  weatherLocation: '铁路沿线',
  heroDeviceName: '1495信号机',
  humidityIncident: {
    title: '环境湿度预警',
    alarmContent: '箱体内部湿度偏高，建议巡检密封状态。',
    analysis: '当前为演示数据，用于联动孪生面板的环境异常提示。',
    suggestion: '建议核查箱体密封、引线接头与排水条件。'
  },
  nearbyStations: {
    main: '1495信号机',
    freight: '1493信号机',
    hs: '1497信号机'
  }
}

export default DEMO_SCENARIO
```

4. 模块：数据与服务层

数据与服务层负责本地模拟数据生成、前端数据访问封装、跨视图状态同步、遥测 WebSocket 连接以及组合式数据调用。

4.1 文件：src/services/api.js

- 文件说明：前端本地内存数据服务，提供信号设备、天气、地震、统计、对话、日志等接口。
- 行数统计：301 行

```
const nowIso = () => new Date().toISOString()

const wait = (ms = 80) => new Promise((resolve) => setTimeout(resolve, ms))

const memory = {
```

```
  signals: [
    {
      id: 1,
      name: '\u4e3b\u4fe1\u53f7\u706f',
      status: 'red',
      temperature: 35.2,
      humidity: 66.4,
      light_intensity: 920,
      voltage: 220.6,
      current: 2.41,
      signal_strength: -47,
    },
  ],
  trainStats: {
    passed_count: 42,
    total_count: 58,
    on_time_rate: 96.5,
    next_train: 'G1502 14:35',
  },
  railway: {
    name: 'XX铁路',
    start_station: 'X站',
    end_station: 'Y站',
    length_km: 255,
  },
  aiConversations: [
    {
      id: 1,
      session_id: 'default',
      role: 'assistant',
      content: '\u7cfb\u7edf\u5df2\u5207\u6362\u5230\u524d\u7aef\u672c\u5730\u6570\u636e\u6a21\u5f0f\u3002',
      created_at: nowIso(),
    },
  ],
  logs: [
    {
      id: 1,
      log_type: 'system',
      log_level: 'info',
      message: 'Frontend-only mode enabled',
      details: null,
      created_at: nowIso(),
    },
  ],
}

const randomFloat = (min, max, precision = 1) => {
  const factor = 10 ** precision
  return Math.round((min + Math.random() * (max - min)) * factor) / factor
}

const generateSeries = (length, min, max, mapValue) => {
  return Array.from({ length }, (_, index) => {
    const value = randomFloat(min, max, 2)
    return mapValue(value, index)
  })
}

const getSeriesLength = (range) => {
  if (range === 'week') return 7
  if (range === 'month') return 30
  return 24
}
```

```
const getSignals = async () => {
  await wait()
  return memory.signals.map((item) => ({ ...item }))
}

const getSignal = async (id) => {
  await wait()
  return memory.signals.find((item) => item.id === Number(id)) || null
}

const createSignal = async (data) => {
  await wait()
  const id = memory.signals.length ? Math.max(...memory.signals.map((item) => item.id)) + 1 : 1
  const created = { id, ...data }
  memory.signals.push(created)
  return created
}

const updateSignal = async (id, data) => {
  await wait()
  const index = memory.signals.findIndex((item) => item.id === Number(id))
  if (index === -1) return null
  memory.signals[index] = { ...memory.signals[index], ...data }
  return memory.signals[index]
}

const deleteSignal = async (id) => {
  await wait()
  const index = memory.signals.findIndex((item) => item.id === Number(id))
  if (index === -1) return { deleted: false }
  memory.signals.splice(index, 1)
  return { deleted: true }
}

const getSeismicData = async (range = '24h') => {
  await wait()
  return generateSeries(getSeriesLength(range), 1.2, 4.2, (value, index) => ({
    id: index + 1,
    level: value,
    location: 'XX区域',
    recorded_at: nowIso(),
  }))
}

const addSeismicData = async (level, location) => {
  await wait()
  return {
    id: Date.now(),
    level,
    location,
    recorded_at: nowIso(),
  }
}

const getWeatherData = async (range = '24h') => {
  await wait()
  return generateSeries(getSeriesLength(range), 16, 34, (value, index) => ({
    id: index + 1,
    temperature: value,
    humidity: Math.round(randomFloat(45, 90, 0)),
    wind_speed: `${randomFloat(1.2, 5.8, 1)}m/s`,
    visibility: `${Math.round(randomFloat(6, 18, 0))}km`,
    pressure: `${Math.round(randomFloat(1002, 1022, 0))}hPa`,
    recorded_at: nowIso(),
```

```
  }))
}

const getCurrentWeather = async () => {
  await wait()
  return {
    icon: '\u26c5',
    temperature: randomFloat(20, 33, 1),
    description: '\u591a\u4e91',
    location: 'XX区域',
    wind_speed: `${randomFloat(2.0, 4.5, 1)}m/s`,
    humidity: `${Math.round(randomFloat(55, 85, 0))}%`,
    visibility: `${Math.round(randomFloat(8, 16, 0))}km`,
    pressure: `${Math.round(randomFloat(1006, 1020, 0))}hPa`,
  }
}

const getAirQuality = async () => {
  await wait()
  const aqi = Math.round(randomFloat(35, 95, 0))
  return {
    aqi,
    level: aqi <= 50 ? '\u4f18' : '\u826f',
    pm25: Math.round(randomFloat(15, 45, 0)),
    pm10: Math.round(randomFloat(30, 80, 0)),
    o3: Math.round(randomFloat(40, 90, 0)),
    no2: Math.round(randomFloat(20, 50, 0)),
  }
}

const getTrainStats = async () => {
  await wait()
  return { ...memory.trainStats }
}

const updateTrainStats = async (data) => {
  await wait()
  memory.trainStats = { ...memory.trainStats, ...data }
  return { ...memory.trainStats }
}

const getRailway = async () => {
  await wait()
  return { ...memory.railway }
}

const getParamHistory = async (type = 'temperature', range = '24h', signalId = 1) => {
  await wait()
  const settings = {
    temperature: { min: 22, max: 45 },
    humidity: { min: 40, max: 90 },
    light: { min: 100, max: 1800 },
    voltage: { min: 205, max: 230 },
    current: { min: 1.4, max: 3.8 },
    signal: { min: -75, max: -40 },
  }
  const { min, max } = settings[type] || settings.temperature

  return generateSeries(getSeriesLength(range), min, max, (value, index) => ({
    id: index + 1,
    signal_id: signalId,
    param_type: type,
    param_value: value,
    created_at: nowIso(),
```

```
  }))
}

const getAiConversations = async (sessionId) => {
  await wait()
  if (!sessionId) return [...memory.aiConversations]
  return memory.aiConversations.filter((item) => item.session_id === sessionId)
}

const addAiConversation = async (sessionId, role, content) => {
  await wait()
  const created = {
    id: Date.now(),
    session_id: sessionId || 'default',
    role,
    content,
    created_at: nowIso(),
  }
  memory.aiConversations.push(created)
  return created
}

const getLogs = async (type, limit = 100) => {
  await wait()
  const source = type ? memory.logs.filter((item) => item.log_type === type) : memory.logs
  return source.slice(-Math.max(1, limit))
}

const addLog = async (logType, logLevel, message, details) => {
  await wait()
  const created = {
    id: Date.now(),
    log_type: logType,
    log_level: logLevel,
    message,
    details: details || null,
    created_at: nowIso(),
  }
  memory.logs.push(created)
  return created
}

const getDashboard = async () => {
  const [weather, airQuality, trainStats, railway, seismic] = await Promise.all([
    getCurrentWeather(),
    getAirQuality(),
    getTrainStats(),
    getRailway(),
    getSeismicData('24h'),
  ])

  return {
    weather,
    airQuality,
    trainStats,
    railway,
    seismic,
  }
}

export {
  getSignals,
  getSignal,
  createSignal,
```

```
  updateSignal,
  deleteSignal,
  getSeismicData,
  addSeismicData,
  getWeatherData,
  getCurrentWeather,
  getAirQuality,
  getTrainStats,
  updateTrainStats,
  getRailway,
  getParamHistory,
  getAiConversations,
  addAiConversation,
  getLogs,
  addLog,
  getDashboard,
}

export default {
  getSignals,
  getSignal,
  createSignal,
  updateSignal,
  deleteSignal,
  getSeismicData,
  addSeismicData,
  getWeatherData,
  getCurrentWeather,
  getAirQuality,
  getTrainStats,
  updateTrainStats,
  getRailway,
  getParamHistory,
  getAiConversations,
  addAiConversation,
  getLogs,
  addLog,
  getDashboard,
}
```

4.2 文件：src/services/mockDataService.js

- 文件说明：大数据面板模拟数据生成器，负责设备、天气、趋势、寿命和告警数据构造。
- 行数统计：775 行

```
/**
 * 大数据面板 Mock 数据服务
 * 提供各种传感器、气象、设备监控等模拟数据
 */

import { severeWeatherEnabled, severeWeatherStartedAt, humidityMitigationAppliedAt, syncedHumidity } from './simulationState.js'

// 随机数生成工具
const random = (min, max, decimal = 0) => {
  const value = Math.random() * (max - min) + min
  return decimal > 0 ? parseFloat(value.toFixed(decimal)) : Math.floor(value)
}

// 可复现的随机数（避免趋势曲线每次刷新都“跳”）
const mulberry32 = (seed) => {
  let t = seed >>> 0
  return () => {
    t += 0x6d2b79f5
    let r = Math.imul(t ^ (t >>> 15), 1 | t)
    r ^= r + Math.imul(r ^ (r >>> 7), 61 | r)
    return ((r ^ (r >>> 14)) >>> 0) / 4294967296
```

```
  }
}

// 随机选择数组元素
const randomChoice = (arr) => arr[random(0, arr.length)]

// 生成时间序列数据
const generateTimeSeries = (length, min, max, decimal = 1) => {
  return Array.from({ length }, () => random(min, max, decimal))
}

// 气象数据
const risingSeries = (length, start, end, jitter = 0.6, decimal = 1) => {
  if (length <= 1) return [Number(end.toFixed(decimal))]
  return Array.from({ length }, (_, i) => {
    const t = i / (length - 1)
    const base = start + (end - start) * t
    // 增加小幅波动：在整体上升的基础上加入轻微正弦扰动
    const wave = Math.sin(t * Math.PI * 4) * (jitter * 0.55)
    return Math.max(0, Math.min(100, random(base + wave - jitter, base + wave + jitter, decimal)))
  })
}

const clamp = (value, min, max) => Math.max(min, Math.min(max, value))

const buildHumidityTrend = ({ seed, baselineEnd, peak, current, phaseT, mode }) => {
  const total = 24
  const rng = mulberry32(seed)

  // baseline：平稳区间（开场应在 20~40 左右）
  const baselineCount = clamp(
    mode === 'falling' ? 10 : 24 - clamp(Math.floor(2 + phaseT * 10), 2, 12),
    0,
    total - 2
  )
  const baseline = Array.from({ length: baselineCount }, (_, idx) => {
    const n = rng()
    const wave = Math.sin((idx / Math.max(1, baselineCount - 1)) * Math.PI * 2) * 0.6
    const v = 22 + n * 8 + wave // 约 22~30
    return Number(clamp(v, 18, 36).toFixed(1))
  })

  if (!baseline.length) baseline.push(Number(clamp(baselineEnd, 18, 36).toFixed(1)))

  const remain = total - baseline.length
  const riseLen = mode === 'falling' ? clamp(Math.floor(6 + phaseT * 6), 6, 12) : remain
  const fallLen = mode === 'falling' ? clamp(remain - riseLen, 4, remain) : 0

  const riseCount = mode === 'falling' ? clamp(riseLen, 2, total - 2) : remain
  const fallCount = mode === 'falling' ? clamp(fallLen, 2, total - riseCount) : 0

  const startRise = baseline[baseline.length - 1]
  const endRise = Number(clamp(peak, 0, 100).toFixed(1))

  const rise = Array.from({ length: riseCount }, (_, idx) => {
    const t = riseCount <= 1 ? 1 : idx / (riseCount - 1)
    // 轻微加速上升：前期更平缓，避免一开场就冲到高值
    const eased = t * t * (3 - 2 * t)
    const v = startRise + (endRise - startRise) * eased
    const jitter = (rng() - 0.5) * 1.2
    return Number(clamp(v + jitter, 0, 100).toFixed(1))
  })

  const fallStart = rise.length ? rise[rise.length - 1] : endRise
```

```
  const fallEnd = Number(clamp(current, 0, 100).toFixed(1))
  const fall = Array.from({ length: fallCount }, (_, idx) => {
    const t = fallCount <= 1 ? 1 : idx / (fallCount - 1)
    const eased = t * t * (3 - 2 * t)
    const v = fallStart + (fallEnd - fallStart) * eased
    const jitter = (rng() - 0.5) * 0.9
    return Number(clamp(v + jitter, 0, 100).toFixed(1))
  })

  let out = [...baseline, ...rise, ...fall]
  out = out.slice(0, total)

  // 强制最后一个点与当前值一致，保证“模拟进行中”时曲线随时间上升/回落
  out[total - 1] = fallEnd

  // 避免刚打开就出现“高湿度尾巴”：把尾部（近 3 个点）限制在当前值附近
  for (let i = total - 3; i < total - 1; i++) {
    if (i < 0) continue
    out[i] = Number(clamp(out[i], 0, fallEnd + 3).toFixed(1))
  }

  return out
}

export const getWeatherData = () => {
  const enabled = Boolean(severeWeatherEnabled.value)

  if (!enabled) {
    return {
      temperature: random(-5, 35, 1),
      humidity: random(21, 25, 1),
      windSpeed: random(0, 25, 1),
      windDirection: randomChoice(['北', '东北', '东', '东南', '南', '西南', '西', '西北']),
      pressure: random(990, 1030, 1),
      visibility: random(1, 30, 1),
      precipitation: random(0, 50, 1),
      uvIndex: random(0, 11),
      airQuality: random(0, 300),
      pm25: random(0, 150),
      pm10: random(0, 200),
      so2: random(0, 50, 2),
      no2: random(0, 100, 2),
      co: random(0, 2, 2),
      o3: random(0, 200, 1),
      weatherType: randomChoice(['晴', '多云', '阴', '小雨', '中雨', '大雨', '雷阵雨', '小雪', '中雪', '大雪', '雾', '霾']),
      temperatureTrend: generateTimeSeries(24, -5, 35, 1),
      humidityTrend: generateTimeSeries(24, 21, 25, 1),
      forecast: Array.from({ length: 7 }, (_, i) => ({
        date: new Date(Date.now() + i * 86400000).toLocaleDateString('zh-CN', { weekday: 'short' }),
        high: random(15, 35),
        low: random(-5, 20),
        weather: randomChoice(['晴', '多云', '阴', '小雨', '中雨', '雷阵雨'])
      }))
    }
  }

  // 恶劣天气演练（Early）：
  // - 湿度在 4s 内进入红色告警（>=70%），随后缓慢上升至高位平台（~92%）
  // - “异常消除”后进入处置回落（约 90s 回落到 35% 左右，进入观察）
  const startedAt = severeWeatherStartedAt.value || Date.now()
  const now = Date.now()
  const elapsed = Math.max(0, now - startedAt)
  const stage1Ms = 4000
  const stage2Ms = 20000
```

```
const base = 22
const redTarget = 75
const peak = 92
const easeOutQuad = (t) => t * (2 - t)
const lerp = (a, b, t) => a + (b - a) * t
const humidityAtTime = (t0) => {
  const e = Math.max(0, Number(t0) || 0)
  if (e <= stage1Ms) {
    const t = easeOutQuad(clamp(e / stage1Ms, 0, 1))
    return Number(lerp(base, redTarget, t).toFixed(1))
  }
  if (e <= stage1Ms + stage2Ms) {
    const t = easeOutQuad(clamp((e - stage1Ms) / stage2Ms, 0, 1))
    return Number(lerp(redTarget, peak, t).toFixed(1))
  }
  return Number(peak.toFixed(1))
}
const riseT = Math.max(0, Math.min(1, elapsed / stage1Ms))

const mitigationAt = humidityMitigationAppliedAt.value
const synced = Number(syncedHumidity.value)
let humidity = Number.isFinite(synced) ? synced : humidityAtTime(elapsed)
let humidityTrend = []

if (Number.isFinite(Number(mitigationAt)) && mitigationAt >= startedAt) {
  const mitigationElapsed = Math.max(0, now - mitigationAt)
  const humidityAtMitigation = humidityAtTime(mitigationAt - startedAt)
  const fallDurationMs = 90000
  const fallT = clamp(mitigationElapsed / fallDurationMs, 0, 1)

  humidityTrend = buildHumidityTrend({
    seed: Math.floor(startedAt / 1000),
    baselineEnd: 24,
    peak: humidityAtMitigation,
    current: humidity,
    phaseT: fallT,
    mode: 'falling'
  })
} else {
  humidityTrend = buildHumidityTrend({
    seed: Math.floor(startedAt / 1000),
    baselineEnd: 24,
    peak: humidity,
    current: humidity,
    phaseT: riseT,
    mode: 'rising'
  })
}

if (Array.isArray(humidityTrend) && humidityTrend.length) {
  humidityTrend[humidityTrend.length - 1] = humidity
}

return {
  temperature: random(12, 28, 1),
  humidity,
  windSpeed: random(10, 25, 1),
  windDirection: randomChoice(['东北', '东', '东南', '北']),
  pressure: random(985, 1008, 1),
  visibility: random(1, 6, 1),
  precipitation: random(20, 50, 1),
  uvIndex: random(0, 3),
  airQuality: random(160, 260),
  pm25: random(110, 180),
```

```
    pm10: random(160, 240),
    so2: random(10, 35, 2),
    no2: random(60, 120, 2),
    co: random(0.6, 1.6, 2),
    o3: random(80, 160, 1),
    weatherType: randomChoice(['大雨', '暴雨', '雷阵雨', '霾']),
    temperatureTrend: generateTimeSeries(24, 10, 28, 1),
    humidityTrend,
    forecast: Array.from({ length: 7 }, (_, i) => ({
      date: new Date(Date.now() + i * 86400000).toLocaleDateString('zh-CN', { weekday: 'short' }),
      high: random(18, 28),
      low: random(10, 18),
      weather: randomChoice(['中雨', '大雨', '雷阵雨', '阴'])
    }))
  }
}

// 传感器数据
export const getSensorData = () => {
  const sensors = []
  const sensorTypes = [
    { type: '温度传感器', unit: '°C', min: -20, max: 60, icon: 'thermometer' },
    { type: '湿度传感器', unit: '%', min: 0, max: 100, icon: 'droplet' },
    { type: '压力传感器', unit: 'MPa', min: 0, max: 10, decimal: 2, icon: 'gauge' },
    { type: '振动传感器', unit: 'mm/s', min: 0, max: 50, decimal: 2, icon: 'activity' },
    { type: '电流传感器', unit: 'A', min: 0, max: 100, decimal: 2, icon: 'zap' },
    { type: '电压传感器', unit: 'V', min: 200, max: 250, decimal: 1, icon: 'bolt' },
    { type: '位移传感器', unit: 'mm', min: 0, max: 100, decimal: 3, icon: 'move' },
    { type: '流量传感器', unit: 'm³/h', min: 0, max: 1000, decimal: 1, icon: 'flow' },
    { type: '转速传感器', unit: 'RPM', min: 0, max: 3000, icon: 'rotate' },
    { type: '噪音传感器', unit: 'dB', min: 20, max: 120, icon: 'volume' },
    { type: '烟雾传感器', unit: 'ppm', min: 0, max: 500, decimal: 1, icon: 'smoke' },
    { type: '光照传感器', unit: 'lux', min: 0, max: 100000, icon: 'sun' }
  ]

  for (let i = 1; i <= 24; i++) {
    const config = sensorTypes[(i - 1) % sensorTypes.length]
    const status = Math.random() > 0.1 ? 'normal' : (Math.random() > 0.5 ? 'warning' : 'error')
    sensors.push({
      id: `SENSOR-${String(i).padStart(3, '0')}`,
      name: `${config.type}-${i}`,
      type: config.type,
      value: random(config.min, config.max, config.decimal || 0),
      unit: config.unit,
      status,
      icon: config.icon,
      location: randomChoice(['A区', 'B区', 'C区', 'D区', '主站', '信号楼', '调度中心']),
      updateTime: new Date().toLocaleTimeString(),
      threshold: {
        min: config.min + (config.max - config.min) * 0.1,
        max: config.max - (config.max - config.min) * 0.1
      },
      history: generateTimeSeries(30, config.min, config.max, config.decimal || 0)
    })
  }
  return sensors
}

// 设备监控数据
export const getEquipmentData = () => {
  const equipments = []
  const equipmentTypes = [
    { type: '信号机', icon: 'signal', count: 45 },
    { type: '道岔', icon: 'switch', count: 32 },
```

```
    { type: '轨道电路', icon: 'track', count: 128 },
    { type: 'UPS电源', icon: 'battery', count: 16 },
    { type: '通信设备', icon: 'network', count: 24 },
    { type: '监控摄像头', icon: 'camera', count: 86 },
    { type: '列车检测器', icon: 'train', count: 18 },
    { type: '防雷设备', icon: 'lightning', count: 42 }
  ]

  equipmentTypes.forEach(({ type, icon, count }) => {
    const normal = random(Math.floor(count * 0.7), count)
    const warning = random(0, count - normal)
    const error = count - normal - warning
    equipments.push({
      type,
      icon,
      total: count,
      normal,
      warning,
      error,
      onlineRate: ((normal / count) * 100).toFixed(1),
      maintenanceCount: random(0, 5),
      avgRunningTime: random(100, 10000)
    })
  })
  return equipments
}

// 告警数据
export const getAlarmData = () => {
  const alarms = []
  const alarmTypes = [
    { level: 'critical', title: '设备严重故障', desc: '信号机XS-012通信中断' },
    { level: 'critical', title: '轨道电路异常', desc: 'GJ-003区段电压异常' },
    { level: 'warning', title: '温度超限预警', desc: '机房温度超过35°C' },
    { level: 'warning', title: 'UPS电量不足', desc: 'UPS-02电量低于20%' },
    { level: 'info', title: '设备维护提醒', desc: '道岔DC-008即将到期检修' },
    { level: 'info', title: '系统升级通知', desc: '计划于今晚22:00进行系统升级' },
    { level: 'warning', title: '网络延迟预警', desc: '主网络延迟超过50ms' },
    { level: 'critical', title: '安全事件', desc: '检测到异常访问尝试' }
  ]

  for (let i = 0; i < 15; i++) {
    const alarm = randomChoice(alarmTypes)
    alarms.push({
      id: `ALM-${Date.now()}-${i}`,
      ...alarm,
      time: new Date(Date.now() - random(0, 86400000)).toLocaleString('zh-CN'),
      source: randomChoice(['A区', 'B区', 'C区', '调度中心', '信号楼']),
      handled: Math.random() > 0.7
    })
  }

  return alarms.sort((a, b) => new Date(b.time) - new Date(a.time))
}

// 列车运行数据
export const getTrainData = () => ({
  totalTrains: random(50, 200),
  runningTrains: random(20, 80),
  delayedTrains: random(0, 10),
  avgDelay: random(0, 15, 1),
  punctualityRate: random(85, 99.9, 1),
  todayTrips: random(200, 500),
  passengerFlow: random(50000, 200000),
```

```
  freightTonnage: random(1000, 10000),
  avgSpeed: random(60, 120, 1),
  trainsByLine: [
    { line: 'XX1线', count: random(10, 30) },
    { line: 'XX2线', count: random(10, 30) },
    { line: 'XX3线', count: random(10, 30) },
    { line: 'XX4线', count: random(10, 30) },
    { line: 'XX5线', count: random(10, 30) }
  ],
  trainsByType: [
    { type: '高铁', count: random(20, 50), color: '#00d4ff' },
    { type: '动车', count: random(15, 40), color: '#00ff88' },
    { type: '普快', count: random(30, 60), color: '#ffaa00' },
    { type: '货运', count: random(40, 80), color: '#ff6b6b' }
  ],
  realtimePositions: Array.from({ length: 20 }, (_, i) => ({
    id: `G${1000 + i}`,
    line: randomChoice(['XX1', 'XX2', 'XX3', 'XX4', 'XX5']),
    position: [random(115, 125, 4), random(30, 45, 4)],
    speed: random(80, 350),
    status: randomChoice(['正常运行', '即将到站', '正在发车', '临时停车'])
  }))
})

// 能耗数据
export const getEnergyData = () => ({
  totalConsumption: random(50000, 100000),
  dailyConsumption: generateTimeSeries(24, 1000, 5000),
  monthlyConsumption: generateTimeSeries(12, 50000, 150000),
  powerLoad: random(1000, 5000, 1),
  peakLoad: random(4000, 6000, 1),
  valleyLoad: random(500, 1500, 1),
  loadRate: random(60, 95, 1),
  energyByType: [
    { type: '牵引供电', value: random(40, 60), color: '#00d4ff' },
    { type: '信号设备', value: random(10, 20), color: '#00ff88' },
    { type: '照明系统', value: random(5, 15), color: '#ffaa00' },
    { type: '空调系统', value: random(15, 25), color: '#ff6b6b' },
    { type: '其他设备', value: random(5, 10), color: '#aa88ff' }
  ],
  carbonEmission: random(100, 500, 1),
  energySavingRate: random(5, 20, 1)
})

// 系统状态数据
export const getSystemStatus = () => ({
  cpuUsage: random(20, 80, 1),
  memoryUsage: random(30, 70, 1),
  diskUsage: random(40, 90, 1),
  networkIn: random(10, 1000, 1),
  networkOut: random(10, 500, 1),
  activeConnections: random(100, 1000),
  requestPerSecond: random(100, 5000),
  avgResponseTime: random(10, 200, 1),
  uptime: random(100, 365),
  services: [
    { name: '主服务器', status: 'running', cpu: random(10, 50, 1), memory: random(20, 60, 1) },
    { name: '数据库服务', status: 'running', cpu: random(10, 40, 1), memory: random(30, 70, 1) },
    { name: '缓存服务', status: 'running', cpu: random(5, 30, 1), memory: random(40, 80, 1) },
    { name: '消息队列', status: 'running', cpu: random(5, 25, 1), memory: random(20, 50, 1) },
    { name: '文件服务', status: Math.random() > 0.1 ? 'running' : 'warning', cpu: random(5, 20, 1), memory: random(10, 40, 1) }
  ],
  recentLogs: Array.from({ length: 10 }, (_, i) => ({
    time: new Date(Date.now() - i * 60000).toLocaleTimeString(),
```

```
    level: randomChoice(['INFO', 'WARN', 'ERROR', 'DEBUG']),
    message: randomChoice([
      '用户登录成功',
      '数据同步完成',
      'API请求超时',
      '数据库连接池已满',
      '缓存命中率: 95.2%',
      '定时任务执行完毕',
      '收到MQTT消息',
      'WebSocket连接建立'
    ])
  }))
})

// 安全监控数据
export const getSecurityData = () => ({
  intrusionAttempts: random(0, 50),
  blockedIPs: random(100, 500),
  activeUsers: random(50, 200),
  failedLogins: random(0, 20),
  securityScore: random(70, 100),
  firewallStatus: 'active',
  lastScanTime: new Date(Date.now() - random(0, 3600000)).toLocaleString('zh-CN'),
  vulnerabilities: {
    critical: random(0, 3),
    high: random(0, 10),
    medium: random(0, 20),
    low: random(0, 50)
  },
  recentEvents: Array.from({ length: 8 }, (_, i) => ({
    time: new Date(Date.now() - i * 1800000).toLocaleTimeString(),
    type: randomChoice(['登录', '登出', '权限变更', '配置修改', '文件访问']),
    user: `user${random(1, 100)}`,
    ip: `${random(1, 255)}.${random(1, 255)}.${random(1, 255)}.${random(1, 255)}`,
    status: randomChoice(['成功', '失败', '待审核'])
  }))
})

// 运维工单数据
export const getWorkOrderData = () => ({
  pending: random(5, 20),
  processing: random(10, 30),
  completed: random(50, 100),
  overdue: random(0, 5),
  avgProcessTime: random(2, 48, 1),
  satisfaction: random(85, 99, 1),
  ordersByType: [
    { type: '设备故障', count: random(10, 30), color: '#ff6b6b' },
    { type: '日常巡检', count: random(20, 50), color: '#00d4ff' },
    { type: '预防维护', count: random(15, 40), color: '#00ff88' },
    { type: '升级改造', count: random(5, 15), color: '#ffaa00' },
    { type: '应急抢修', count: random(0, 10), color: '#aa88ff' }
  ],
  recentOrders: Array.from({ length: 8 }, (_, i) => ({
    id: `WO-${Date.now()}-${i}`,
    title: randomChoice([
      '信号机故障维修',
      '道岔润滑保养',
      'UPS电池更换',
      '摄像头清洁',
      '网络设备巡检',
      '软件版本升级',
      '安全漏洞修复',
      '设备参数调整'
```

```
    ]),
    priority: randomChoice(['紧急', '高', '中', '低']),
    assignee: `工程师${random(1, 20)}`,
    status: randomChoice(['待处理', '处理中', '已完成', '已关闭']),
    createTime: new Date(Date.now() - random(0, 86400000 * 3)).toLocaleString('zh-CN')
  }))
})

// 统计概览数据
export const getOverviewData = () => ({
  totalDevices: 391,
  onlineDevices: random(350, 390),
  alarmCount: random(5, 30),
  todayEvents: random(100, 500),
  dataPoints: random(1000000, 10000000),
  systemHealth: random(90, 99, 1),
  dataQuality: random(95, 99.9, 1),
  predictionAccuracy: random(85, 98, 1),
  keyMetrics: [
    { label: '设备在线率', value: random(95, 99.9, 1), unit: '%', trend: random(-2, 2, 1), color: '#00d4ff' },
    { label: '信号正常率', value: random(98, 99.9, 1), unit: '%', trend: random(-1, 1, 1), color: '#00ff88' },
    { label: '故障响应时间', value: random(5, 30, 0), unit: '分钟', trend: random(-5, 5, 1), color: '#ffaa00' },
    { label: '系统可用性', value: random(99, 99.99, 2), unit: '%', trend: random(-0.1, 0.1, 2), color: '#ff6b6b' }
  ],
  trendData: {
    week: generateTimeSeries(7, 80, 100, 1),
    month: generateTimeSeries(30, 80, 100, 1),
    year: generateTimeSeries(12, 80, 100, 1)
  }
})

// 地图热点数据
export const getMapHotspots = () => Array.from({ length: 20 }, (_, i) => ({
  id: i + 1,
  name: randomChoice(['X站', 'Y站', 'X北站', 'X东站', 'Y北站', 'Y东站', 'X信号所', 'Y信号所', 'XX线路所', 'XX货场']),
  position: [random(100, 130, 4), random(20, 45, 4)],
  value: random(10, 100),
  status: randomChoice(['正常', '繁忙', '拥堵', '维护中']),
  trains: random(5, 30),
  passengers: random(1000, 50000)
}))

// 信号调度系统数据
export const getSignalDispatchData = () => {
  // 线路定义
  const lines = [
    { id: 'L1', name: 'XX1线', color: '#00d4ff', length: 1318 },
    { id: 'L2', name: 'XX2线', color: '#00ff88', length: 2298 },
    { id: 'L3', name: 'XX3线', color: '#ffaa00', length: 2252 },
    { id: 'L4', name: 'XX4线', color: '#ff6b6b', length: 1759 },
    { id: 'L5', name: 'XX5线', color: '#aa88ff', length: 1249 }
  ]

  // 信号机状态
  const signalMachines = []
  const signalStates = ['开放', '关闭', '故障', '维护']
  const signalColors = { '开放': '#00ff88', '关闭': '#ff6b6b', '故障': '#ffaa00', '维护': '#888' }

  for (let i = 1; i <= 32; i++) {
    const state = randomChoice(signalStates)
    signalMachines.push({
      id: `XH-${String(i).padStart(3, '0')}`,
      name: `信号机${i}`,
      line: randomChoice(lines).id,
```

```
    state,
    color: signalColors[state],
    position: { x: random(50, 750), y: random(50, 350) },
    direction: randomChoice(['上行', '下行']),
    lastChange: new Date(Date.now() - random(0, 3600000)).toLocaleTimeString()
  })
}

// 道岔状态
const switches = []
const switchStates = ['定位', '反位', '四开', '故障']

for (let i = 1; i <= 24; i++) {
  const state = randomChoice(switchStates)
  switches.push({
    id: `DC-${String(i).padStart(3, '0')}`,
    name: `道岔${i}`,
    line: randomChoice(lines).id,
    state,
    locked: Math.random() > 0.3,
    position: { x: random(50, 750), y: random(50, 350) },
    operationCount: random(10, 200)
  })
}

// 轨道区段
const trackSections = []
const trackStates = ['空闲', '占用', '锁闭', '故障']

for (let i = 1; i <= 48; i++) {
  const state = randomChoice(trackStates)
  trackSections.push({
    id: `GJ-${String(i).padStart(3, '0')}`,
    name: `区段${i}`,
    line: randomChoice(lines).id,
    state,
    voltage: random(15, 25, 1),
    length: random(500, 2000),
    occupied: state === '占用',
    trainId: state === '占用' ? `G${random(1000, 9999)}` : null
  })
}

// 实时列车调度信息
const trainDispatch = []
const trainStates = ['正常运行', '等待信号', '减速运行', '临时停车', '晚点运行']

for (let i = 1; i <= 15; i++) {
  const line = randomChoice(lines)
  trainDispatch.push({
    id: `G${1000 + i}`,
    line: line.id,
    lineName: line.name,
    lineColor: line.color,
    state: randomChoice(trainStates),
    position: { x: random(50, 750), y: random(50, 350) },
    speed: random(80, 350),
    direction: randomChoice(['上行', '下行']),
    nextStation: randomChoice(['北京南', '上海虹桥', '广州南', '武汉', '郑州东']),
    eta: random(5, 60),
    delay: random(-5, 15),
    signalAhead: randomChoice(['绿灯', '黄灯', '红灯', '双黄灯']),
    priority: random(1, 5)
  })
```

```
}

// 调度命令
const dispatchCommands = []
const commandTypes = ['发车', '停车', '变更进路', '临时限速', '恢复常速', '越站', '扣车']

for (let i = 0; i < 8; i++) {
  dispatchCommands.push({
    id: `CMD-${Date.now()}-${i}`,
    type: randomChoice(commandTypes),
    trainId: `G${random(1000, 9999)}`,
    content: randomChoice([
      '准许从北京南站发车',
      '在济南西站临时停车',
      '变更进路至3道',
      '限速120km/h通过',
      '恢复正常速度运行',
      '越过南京南站不停车',
      '在站扣车等待'
    ]),
    status: randomChoice(['已执行', '执行中', '待执行', '已取消']),
    issuer: `调度员${random(1, 10)}`,
    time: new Date(Date.now() - random(0, 3600000)).toLocaleTimeString()
  })
}

// 进路信息
const routes = []
for (let i = 1; i <= 12; i++) {
  routes.push({
    id: `ROUTE-${String(i).padStart(2, '0')}`,
    name: `进路${i}`,
    from: randomChoice(['北京南', '上海虹桥', '广州南', '武汉', '郑州东', '西安北']),
    to: randomChoice(['北京南', '上海虹桥', '广州南', '武汉', '郑州东', '西安北']),
    status: randomChoice(['已建立', '已锁闭', '已解锁', '待建立']),
    signals: [`XH-${random(1, 32).toString().padStart(3, '0')}`, `XH-${random(1, 32).toString().padStart(3, '0')}`],
    switches: [`DC-${random(1, 24).toString().padStart(3, '0')}`, `DC-${random(1, 24).toString().padStart(3, '0')}`],
    sections: [`GJ-${random(1, 48).toString().padStart(3, '0')}`, `GJ-${random(1, 48).toString().padStart(3, '0')}`]
  })
}

return {
  lines,
  signalMachines,
  switches,
  trackSections,
  trainDispatch,
  dispatchCommands,
  routes,
  stats: {
    totalSignals: 32,
    openSignals: signalMachines.filter(s => s.state === '开放').length,
    closedSignals: signalMachines.filter(s => s.state === '关闭').length,
    faultSignals: signalMachines.filter(s => s.state === '故障').length,
    totalSwitches: 24,
    normalSwitches: switches.filter(s => s.state !== '故障').length,
    faultSwitches: switches.filter(s => s.state === '故障').length,
    totalSections: 48,
    occupiedSections: trackSections.filter(s => s.state === '占用').length,
    freeSections: trackSections.filter(s => s.state === '空闲').length,
    lockedSections: trackSections.filter(s => s.state === '锁闭').length,
    runningTrains: trainDispatch.filter(t => t.state === '正常运行').length,
    delayedTrains: trainDispatch.filter(t => t.delay > 0).length,
    activeRoutes: routes.filter(r => r.status === '已建立' || r.status === '已锁闭').length
```

```
    }
  }
}

// 信号设备分布（饼图数据）
export const getSignalDistribution = () => ({
  signalStatus: [
    { name: '开放', value: random(20, 28), color: '#00ff88' },
    { name: '关闭', value: random(2, 5), color: '#ff6b6b' },
    { name: '故障', value: random(0, 2), color: '#ffaa00' },
    { name: '维护', value: random(0, 2), color: '#888888' }
  ],
  switchStatus: [
    { name: '定位', value: random(15, 20), color: '#00d4ff' },
    { name: '反位', value: random(4, 8), color: '#00ff88' },
    { name: '四开', value: random(0, 2), color: '#ffaa00' },
    { name: '故障', value: random(0, 1), color: '#ff6b6b' }
  ],
  trackStatus: [
    { name: '空闲', value: random(30, 40), color: '#00ff88' },
    { name: '占用', value: random(5, 12), color: '#ff6b6b' },
    { name: '锁闭', value: random(2, 6), color: '#ffaa00' },
    { name: '故障', value: random(0, 2), color: '#aa88ff' }
  ],
  trainStatus: [
    { name: '正常运行', value: random(8, 12), color: '#00ff88' },
    { name: '等待信号', value: random(1, 3), color: '#ffaa00' },
    { name: '减速运行', value: random(0, 2), color: '#00d4ff' },
    { name: '临时停车', value: random(0, 1), color: '#ff6b6b' }
  ],
  lineLoad: [
    { name: 'XX1线', value: random(25, 35), color: '#00d4ff' },
    { name: 'XX2线', value: random(20, 30), color: '#00ff88' },
    { name: 'XX3线', value: random(15, 25), color: '#ffaa00' },
    { name: 'XX4线', value: random(10, 20), color: '#ff6b6b' },
    { name: 'XX5线', value: random(10, 20), color: '#aa88ff' }
  ]
})

// 调度效率统计
export const getDispatchEfficiency = () => ({
  // 每小时调度列车数
  hourlyDispatch: generateTimeSeries(24, 15, 45),
  // 调度成功率
  successRate: random(95, 99.9, 1),
  // 平均响应时间
  avgResponseTime: random(3, 15, 1),
  // 进路建立时间
  avgRouteTime: random(5, 20, 1),
  // 调度命令执行率
  commandExecutionRate: random(90, 99, 1),
  // 今日调度统计
  todayStats: {
    totalCommands: random(200, 500),
    executedCommands: random(180, 480),
    cancelledCommands: random(5, 20),
    pendingCommands: random(5, 30),
    avgDelay: random(0, 8, 1),
    punctualityRate: 100
  },
  // 近7天趋势
  weeklyTrend: Array.from({ length: 7 }, (_, i) => ({
    date: new Date(Date.now() - (6 - i) * 86400000).toLocaleDateString('zh-CN', { month: 'numeric', day: 'numeric' }),
    trains: random(300, 500),
```

```
    commands: random(400, 700),
    successRate: random(94, 99, 1)
  }))
})

// 实时数据更新（模拟）
export const subscribeToRealtimeData = (callback, interval = 3000) => {
  const timer = setInterval(() => {
    callback({
      timestamp: Date.now(),
      weather: {
        temperature: random(-5, 35, 1),
        humidity: random(30, 95, 1),
        windSpeed: random(0, 25, 1)
      },
      system: {
        cpuUsage: random(20, 80, 1),
        memoryUsage: random(30, 70, 1),
        networkIn: random(10, 1000, 1),
        networkOut: random(10, 500, 1)
      },
      sensors: getSensorData().slice(0, 6).map(s => ({
        id: s.id,
        value: s.value,
        status: s.status
      }))
    })
  }, interval)

  return () => clearInterval(timer)
}

export default {
  getWeatherData,
  getSensorData,
  getEquipmentData,
  getAlarmData,
  getTrainData,
  getEnergyData,
  getSystemStatus,
  getSecurityData,
  getWorkOrderData,
  getOverviewData,
  getMapHotspots,
  getSignalDispatchData,
  getSignalDistribution,
  getDispatchEfficiency,
  subscribeToRealtimeData
}
```

4.3 文件：src/services/simulationState.js

- 文件说明：跨页面共享的恶劣天气、湿度告警和处置状态同步模块。
- 行数统计：114 行

```
import { ref } from 'vue'

// 全局模拟状态（用于跨页面共享）
export const severeWeatherEnabled = ref(false)
export const severeWeatherStartedAt = ref(null)
export const humidityMitigationAppliedAt = ref(null)
export const syncedHumidity = ref(22)

// 大数据面板：湿度异常告警弹窗的全局状态（用于跨页面切换保持一致）
export const humidityAlertDismissed = ref(false)
export const humidityAlertSuppressed = ref(false)
```

```
export const humidityAlertProcessing = ref(false)

let humiditySyncTimer = null

const clamp = (value, min, max) => Math.max(min, Math.min(max, value))

const computeSyncedSevereHumidity = (nowMs = Date.now()) => {
  const startedAt = Number(severeWeatherStartedAt.value) || nowMs
  const now = Number(nowMs) || Date.now()

  // Early 天气演练：湿度 4 秒内进入红色告警（>=70%），随后再缓慢爬升到高位平台
  const stage1Ms = 4000
  const stage2Ms = 20000
  const base = 22
  const redTarget = 75
  const peak = 92

  const easeOutQuad = (t) => t * (2 - t)
  const lerp = (a, b, t) => a + (b - a) * t

  const humidityAtTime = (elapsedMs) => {
    const e = Math.max(0, Number(elapsedMs) || 0)
    if (e <= stage1Ms) {
      const t = easeOutQuad(clamp(e / stage1Ms, 0, 1))
      return Number(lerp(base, redTarget, t).toFixed(1))
    }
    if (e <= stage1Ms + stage2Ms) {
      const t = easeOutQuad(clamp((e - stage1Ms) / stage2Ms, 0, 1))
      return Number(lerp(redTarget, peak, t).toFixed(1))
    }
    return Number(peak.toFixed(1))
  }

  const mitigationAt = Number(humidityMitigationAppliedAt.value)
  const elapsed = Math.max(0, now - startedAt)

  if (Number.isFinite(mitigationAt) && mitigationAt >= startedAt) {
    const mitigationElapsed = Math.max(0, now - mitigationAt)
    const humidityAtMitigation = humidityAtTime(mitigationAt - startedAt)
    const fallDurationMs = 90000
    const fallT = clamp(mitigationElapsed / fallDurationMs, 0, 1)
    const target = 35
    const fallHumidity = humidityAtMitigation + (target - humidityAtMitigation) * fallT
    return Number(clamp(fallHumidity, 0, 100).toFixed(1))
  }

  return humidityAtTime(elapsed)
}

const tickSyncedHumidity = () => {
  if (!severeWeatherEnabled.value) return
  syncedHumidity.value = computeSyncedSevereHumidity(Date.now())
}

const startHumiditySync = () => {
  if (humiditySyncTimer) return
  tickSyncedHumidity()
  humiditySyncTimer = setInterval(tickSyncedHumidity, 250)
}

const stopHumiditySync = () => {
  if (!humiditySyncTimer) return
  clearInterval(humiditySyncTimer)
  humiditySyncTimer = null
```

```
}

export const markHumidityMitigationApplied = () => {
  humidityMitigationAppliedAt.value = Date.now()
  tickSyncedHumidity()
}

export const resetHumidityMitigation = () => {
  humidityMitigationAppliedAt.value = null
  tickSyncedHumidity()
}

export const setHumidityAlertDismissed = (dismissed) => {
  humidityAlertDismissed.value = Boolean(dismissed)
}

export const setHumidityAlertSuppressed = (suppressed) => {
  humidityAlertSuppressed.value = Boolean(suppressed)
}

export const setHumidityAlertProcessing = (processing) => {
  humidityAlertProcessing.value = Boolean(processing)
}

export const setSevereWeatherEnabled = (enabled) => {
  const next = Boolean(enabled)
  severeWeatherEnabled.value = next
  severeWeatherStartedAt.value = next ? Date.now() : null
  resetHumidityMitigation()
  setHumidityAlertDismissed(false)
  setHumidityAlertSuppressed(false)
  setHumidityAlertProcessing(false)
  if (next) {
    startHumiditySync()
    return
  }
  stopHumiditySync()
}
```

4.4 文件：src/services/telemetrySocket.js

- 文件说明：浏览器端 WebSocket 遥测连接、重连与最新数据缓存模块。
- 行数统计：115 行

```
import { ref } from 'vue'

const defaultUrl = () => {
  const envUrl = import.meta?.env?.VITE_TELEMETRY_WS_URL
  if (typeof envUrl === 'string' && envUrl.trim()) return envUrl.trim()
  return 'ws://localhost:8080/ws'
}

export const telemetryConnected = ref(false)
export const lastTelemetry = ref(null)
export const lastTelemetryAt = ref(0)

let socket = null
let reconnectTimer = null
let reconnectAttempt = 0

const scheduleReconnect = () => {
  if (reconnectTimer) return
  const base = 600
  const max = 8000
  const delay = Math.min(max, base * Math.pow(1.6, reconnectAttempt++))
  reconnectTimer = setTimeout(() => {
```

```
    reconnectTimer = null
    connectTelemetry()
  }, delay)
}

export const connectTelemetry = () => {
  const url = defaultUrl()
  try {
    if (socket && (socket.readyState === WebSocket.OPEN || socket.readyState === WebSocket.CONNECTING)) {
      return socket
    }

    socket = new WebSocket(url)

    socket.addEventListener('open', () => {
      telemetryConnected.value = true
      reconnectAttempt = 0
      console.info('[telemetry] ws open', url)
    })

    socket.addEventListener('close', (ev) => {
      telemetryConnected.value = false
      console.warn('[telemetry] ws close', { code: ev?.code, reason: ev?.reason })
      scheduleReconnect()
    })

    socket.addEventListener('error', () => {
      telemetryConnected.value = false
      console.warn('[telemetry] ws error')
      try {
        socket?.close()
      } catch (_) {
        // ignore
      }
    })

    socket.addEventListener('message', (ev) => {
      const text = typeof ev?.data === 'string' ? ev.data : ''
      if (!text) return

      try {
        const parsed = JSON.parse(text)
        if (parsed?.topic === 'telemetry' && parsed?.data) {
          lastTelemetry.value = parsed.data
          lastTelemetryAt.value = Date.now()
          const d = parsed.data
          console.debug('[telemetry] recv', {
            device: d?.device,
            device_id: d?.device_id,
            type: d?.type,
            ts_ms: d?.ts_ms,
            distance_m: d?.distance_m,
            distance_cm: d?.distance_cm,
            water_active: d?.water_active
          })
        }
      } catch (_) {
        // ignore non-json
      }
    })

    return socket
  } catch (_) {
    telemetryConnected.value = false
```

```
    scheduleReconnect()
    return null
  }
}

export const disconnectTelemetry = () => {
  if (reconnectTimer) {
    clearTimeout(reconnectTimer)
    reconnectTimer = null
  }
  reconnectAttempt = 0
  telemetryConnected.value = false
  try {
    socket?.close()
  } catch (_) {
    // ignore
  } finally {
    socket = null
  }
}

export default {
  telemetryConnected,
  lastTelemetry,
  lastTelemetryAt,
  connectTelemetry,
  disconnectTelemetry
}
```

4.5 文件：src/composables/useApiData.js

- 文件说明：组合式 API 数据调用封装，统一 loading、error 与数据状态。
- 行数统计：239 行

```
// 数据获取组合函数
import { ref } from 'vue'
import api from '../services/api.js'

// 获取仪表盘汇总数据
export const useDashboardData = () => {
  const loading = ref(false)
  const error = ref(null)
  const dashboardData = ref(null)

  const fetchData = async () => {
    loading.value = true
    error.value = null
    try {
      dashboardData.value = await api.getDashboard()
    } catch (e) {
      error.value = e.message
      console.error('获取仪表盘数据失败:', e)
    } finally {
      loading.value = false
    }
  }

  return { loading, error, dashboardData, fetchData }
}

// 获取信号灯数据
export const useSignalsData = () => {
  const loading = ref(false)
  const error = ref(null)
  const signals = ref([])
```

```
  const fetchSignals = async () => {
    loading.value = true
    error.value = null
    try {
      signals.value = await api.getSignals()
    } catch (e) {
      error.value = e.message
      console.error('获取信号灯数据失败:', e)
    } finally {
      loading.value = false
    }
  }

  return { loading, error, signals, fetchSignals }
}

// 获取地震数据
export const useSeismicData = () => {
  const loading = ref(false)
  const error = ref(null)
  const seismicData = ref([])

  const fetchSeismic = async (range = '24h') => {
    loading.value = true
    error.value = null
    try {
      seismicData.value = await api.getSeismicData(range)
    } catch (e) {
      error.value = e.message
      console.error('获取地震数据失败:', e)
    } finally {
      loading.value = false
    }
  }

  return { loading, error, seismicData, fetchSeismic }
}

// 获取天气数据
export const useWeatherData = () => {
  const loading = ref(false)
  const error = ref(null)
  const weatherData = ref([])
  const currentWeather = ref(null)

  const fetchWeather = async (range = '24h') => {
    loading.value = true
    error.value = null
    try {
      weatherData.value = await api.getWeatherData(range)
    } catch (e) {
      error.value = e.message
      console.error('获取天气数据失败:', e)
    } finally {
      loading.value = false
    }
  }

  const fetchCurrentWeather = async () => {
    loading.value = true
    error.value = null
    try {
      currentWeather.value = await api.getCurrentWeather()
    } catch (e) {
```

```
      error.value = e.message
      console.error('获取当前天气失败:', e)
    } finally {
      loading.value = false
    }
  }

  return { loading, error, weatherData, currentWeather, fetchWeather, fetchCurrentWeather }
}

// 获取空气质量数据
export const useAirQualityData = () => {
  const loading = ref(false)
  const error = ref(null)
  const airQuality = ref(null)

  const fetchAirQuality = async () => {
    loading.value = true
    error.value = null
    try {
      airQuality.value = await api.getAirQuality()
    } catch (e) {
      error.value = e.message
      console.error('获取空气质量失败:', e)
    } finally {
      loading.value = false
    }
  }

  return { loading, error, airQuality, fetchAirQuality }
}

// 获取列车统计数据
export const useTrainStatsData = () => {
  const loading = ref(false)
  const error = ref(null)
  const trainStats = ref(null)

  const fetchTrainStats = async () => {
    loading.value = true
    error.value = null
    try {
      trainStats.value = await api.getTrainStats()
    } catch (e) {
      error.value = e.message
      console.error('获取列车统计失败:', e)
    } finally {
      loading.value = false
    }
  }

  return { loading, error, trainStats, fetchTrainStats }
}

// 获取铁道信息
export const useRailwayData = () => {
  const loading = ref(false)
  const error = ref(null)
  const railway = ref(null)

  const fetchRailway = async () => {
    loading.value = true
    error.value = null
    try {
```

```
      railway.value = await api.getRailway()
    } catch (e) {
      error.value = e.message
      console.error('获取铁道信息失败:', e)
    } finally {
      loading.value = false
    }
  }

  return { loading, error, railway, fetchRailway }
}

// 获取参数历史数据
export const useParamsHistory = () => {
  const loading = ref(false)
  const error = ref(null)
  const paramHistory = ref([])

  const fetchParamHistory = async (type = 'temperature', range = '24h', signalId = 1) => {
    loading.value = true
    error.value = null
    try {
      paramHistory.value = await api.getParamHistory(type, range, signalId)
    } catch (e) {
      error.value = e.message
      console.error('获取参数历史失败:', e)
    } finally {
      loading.value = false
    }
  }

  return { loading, error, paramHistory, fetchParamHistory }
}

// 获取AI对话数据
export const useAiConversations = () => {
  const loading = ref(false)
  const error = ref(null)
  const conversations = ref([])

  const fetchConversations = async (sessionId = null) => {
    loading.value = true
    error.value = null
    try {
      conversations.value = await api.getAiConversations(sessionId)
    } catch (e) {
      error.value = e.message
      console.error('获取AI对话失败:', e)
    } finally {
      loading.value = false
    }
  }

  const addConversation = async (sessionId, role, content) => {
    try {
      const newConv = await api.addAiConversation(sessionId, role, content)
      conversations.value.push(newConv)
      return newConv
    } catch (e) {
      console.error('添加AI对话失败:', e)
      throw e
    }
  }
```

```
  return { loading, error, conversations, fetchConversations, addConversation }
}

export default {
  useDashboardData,
  useSignalsData,
  useSeismicData,
  useWeatherData,
  useAirQualityData,
  useTrainStatsData,
  useRailwayData,
  useParamsHistory,
  useAiConversations
}
```

5. 模块：表现层

表现层包括三大主视图及其子面板组件，用于实现 Cesium 地理可视化、Three.js 数字孪生展示和大数据可视化界面。

5.1 文件：src/components/CesiumView.vue

- 文件说明：Cesium 地理信息可视化主界面，含地图、巡航、气象、面板、AI 对话和缓存逻辑。
- 行数统计：2823 行

```
<template>
  <div class="cesium-view">
    <!-- Cesium 容器 -->
    <div id="cesiumContainer" ref="cesiumContainer"></div>

    <div v-if="!networkOnline" class="offline-badge">离线：使用缓存数据</div>

    <!-- 控制按钮 -->
    <div class="control-buttons">
      <button class="reset-btn" @click="resetView">🎯 复位视角</button>
      <button class="toggle-btn" @click="toggleLeftPanel">
        {{ leftPanelVisible ? '◀ 隐藏左侧' : '▶ 显示左侧' }}
      </button>
      <button class="toggle-btn" @click="toggleRightPanel">
        {{ rightPanelVisible ? '隐藏右侧 ▶' : '◀ 显示右侧' }}
      </button>
      <button class="toggle-btn" :class="{ active: cameraCruiseEnabled }" @click="toggleCameraCruise">
        {{ cameraCruiseEnabled ? '📹 停止巡航' : '📹 自动巡航' }}
      </button>
      <button class="toggle-btn" :class="{ active: severeWeatherEnabled }" @click="toggleSevereWeather">
        {{ severeWeatherEnabled ? '⛈️ 停止恶劣天气' : '⛈️ 恶劣天气模拟' }}
      </button>
    </div>

    <!-- 左侧面板 - 地理信息与铁道数据 -->
    <transition name="slide-left">
      <div v-show="leftPanelVisible" class="side-panel left-panel">
        <div class="panel-card">
          <div class="panel-header">
            <span class="panel-title">📡 地理预告</span>
            <span class="panel-subtitle">GEO BULLETIN</span>
          </div>
          <div class="panel-content">
            <div class="geo-ticker-shell">
              <div class="geo-ticker-track">
                <span
                  v-for="(item, idx) in geoTickerItemsLoop"
                  :key="`${idx}-${item}`"
                  class="geo-ticker-item"
                >
                  {{ item }}
                </span>
              </div>
```

```
      </div>
    </div>
  </div>

  <!-- 地理震动监测 -->
  <div class="panel-card">
    <div class="panel-header">
      <span class="panel-title">📍 地理震动监测</span>
      <span class="panel-subtitle">SEISMIC MONITORING</span>
    </div>
    <div class="panel-content">
      <div class="seismic-display">
        <div class="seismic-gauge">
          <svg viewBox="0 0 120 80" class="gauge-svg">
            <rect x="10" y="60" width="100" height="15" rx="3" fill="rgba(0,200,255,0.1)" />
            <rect x="10" y="60" :width="seismicLevel * 10" height="15" rx="3" :fill="seismicColor" />
            <line x1="35" y1="55" x2="35" y2="80" stroke="rgba(255,255,255,0.3)" stroke-width="1" />
            <line x1="60" y1="55" x2="60" y2="80" stroke="rgba(255,255,255,0.3)" stroke-width="1" />
            <line x1="85" y1="55" x2="85" y2="80" stroke="rgba(255,255,255,0.3)" stroke-width="1" />
          </svg>
        </div>
        <div class="seismic-info">
          <div class="seismic-value">
            <span class="value-label">震动等级</span>
            <span class="value-num" :style="{ color: seismicColor }">{{ seismicLevel.toFixed(1) }}</span>
          </div>
          <div class="seismic-status" :class="seismicStatusClass">
            {{ seismicStatus }}
          </div>
        </div>
      </div>
      <div class="vibration-history">
        <div class="history-header">
          <span class="history-label">震动曲线</span>
          <select v-model="seismicTimeRange" class="time-select" @change="onSeismicTimeChange">
            <option value="24h">近24小时</option>
            <option value="week">近一周</option>
            <option value="month">近一个月</option>
          </select>
        </div>
        <svg viewBox="0 0 280 60" class="history-svg">
          <defs>
            <linearGradient id="seismicGradient" x1="0%" y1="0%" x2="0%" y2="100%">
              <stop offset="0%" style="stop-color:#00d4ff;stop-opacity:0.3" />
              <stop offset="100%" style="stop-color:#00d4ff;stop-opacity:0" />
            </linearGradient>
          </defs>
          <polygon :points="seismicAreaPoints" fill="url(#seismicGradient)" />
          <polyline :points="seismicLinePoints" fill="none" stroke="#00d4ff" stroke-width="2" />
          <circle v-for="(point, idx) in seismicChartPoints" :key="idx"
            :cx="point.x" :cy="point.y" r="3" fill="#00d4ff"
            @mouseenter="hoveredSeismicPoint = idx"
            @mouseleave="hoveredSeismicPoint = null" />
        </svg>
        <div v-if="hoveredSeismicPoint !== null" class="chart-tooltip">
          {{ seismicTimeLabels[hoveredSeismicPoint] }}: {{ currentSeismicData[hoveredSeismicPoint]?.toFixed(1) }}
        </div>
      </div>
    </div>
  </div>

  <!-- 当前位置信息 -->
  <div class="panel-card">
    <div class="panel-header">
```

```
.alarm-tab:hover {
  background: rgba(0, 100, 150, 0.3);
  color: #00d4ff;
}

.alarm-tab.active {
  background: rgba(0, 200, 255, 0.2);
  border-color: rgba(0, 200, 255, 0.5);
  color: #00d4ff;
}

.alarm-list {
  display: flex;
  flex-direction: column;
  gap: 8px;
  max-height: 250px;
  overflow-y: auto;
}

.alarm-item {
  display: flex;
  gap: 10px;
  padding: 10px;
  background: rgba(0, 50, 100, 0.2);
  border-radius: 8px;
  border-left: 3px solid;
  transition: all 0.3s;
}

.alarm-item.critical { border-left-color: #ff6b6b; }
.alarm-item.warning { border-left-color: #ffaa00; }
.alarm-item.info { border-left-color: #00d4ff; }
.alarm-item.handled { opacity: 0.6; }

.alarm-content {
  flex: 1;
  min-width: 0;
}

.alarm-title {
  font-size: 15px;
  color: #fff;
  margin-bottom: 3px;
}

.alarm-desc {
  font-size: 12px;
  color: #aaa;
  margin-bottom: 5px;
}

.alarm-meta {
  display: flex;
  gap: 15px;
  font-size: 11px;
  color: #666;
}

.alarm-actions {
  display: flex;
  align-items: center;
}

.action-btn {
```

```
  padding: 4px 10px;
  background: rgba(0, 200, 255, 0.2);
  border: 1px solid rgba(0, 200, 255, 0.4);
  border-radius: 4px;
  color: #00d4ff;
  font-size: 12px;
  cursor: pointer;
  transition: all 0.3s;
}

.action-btn:hover {
  background: rgba(0, 200, 255, 0.3);
}

.handled-tag {
  font-size: 12px;
  color: #00ff88;
}

/* 安全监控 */
.security-overview {
  display: flex;
  gap: 15px;
  margin-bottom: 15px;
}

.security-score {
  width: 80px;
  height: 80px;
}

.score-ring {
  width: 100%;
  height: 100%;
}

.score-circle {
  transform: rotate(-90deg);
  transform-origin: center;
  transition: stroke-dashoffset 1s ease;
}

.security-stats {
  flex: 1;
  display: flex;
  flex-direction: column;
  justify-content: center;
  gap: 8px;
}

.sec-stat {
  display: flex;
  justify-content: space-between;
  align-items: center;
}

.sec-value {
  font-size: 20px;
  font-weight: bold;
  color: #fff;
}

.sec-label {
  font-size: 12px;
```

```
  color: #888;
}

/* 漏洞统计 */
.vulnerability-stats {
  margin-bottom: 15px;
}

.vuln-title, .events-title, .orders-title {
  font-size: 13px;
  color: #888;
  margin-bottom: 8px;
}

.vuln-bars {
  display: flex;
  flex-direction: column;
  gap: 6px;
}

.vuln-item {
  display: flex;
  align-items: center;
  gap: 8px;
}

.vuln-label {
  font-size: 10px;
  color: #888;
  width: 30px;
}

.vuln-bar {
  flex: 1;
  height: 4px;
  background: rgba(255, 255, 255, 0.1);
  border-radius: 2px;
  overflow: hidden;
}

.vuln-fill {
  height: 100%;
  border-radius: 2px;
  transition: width 0.5s ease;
}

.vuln-item.critical .vuln-fill { background: #ff6b6b; }
.vuln-item.high .vuln-fill { background: #ff9966; }
.vuln-item.medium .vuln-fill { background: #ffaa00; }
.vuln-item.low .vuln-fill { background: #00d4ff; }

.vuln-count {
  font-size: 10px;
  color: #aaa;
  width: 20px;
  text-align: right;
}

/* 安全事件 */
.event-list {
  display: flex;
  flex-direction: column;
  gap: 5px;
}
```

```
.event-item {
  display: flex;
  align-items: center;
  gap: 10px;
  padding: 6px 8px;
  background: rgba(0, 50, 100, 0.2);
  border-radius: 4px;
  font-size: 12px;
}

.event-type {
  color: #00d4ff;
}

.event-user {
  color: #888;
  flex: 1;
}

.event-status {
  padding: 2px 6px;
  border-radius: 3px;
}

.event-status.成功 { background: rgba(0, 255, 136, 0.2); color: #00ff88; }
.event-status.失败 { background: rgba(255, 107, 107, 0.2); color: #ff6b6b; }
.event-status.待审核 { background: rgba(255, 170, 0, 0.2); color: #ffaa00; }

/* 运维工单 */
.workorder-stats {
  display: flex;
  justify-content: space-around;
  margin-bottom: 15px;
}

.wo-stat {
  text-align: center;
}

.wo-value {
  font-size: 24px;
  font-weight: bold;
  display: block;
}

.wo-stat.pending .wo-value { color: #ffaa00; }
.wo-stat.processing .wo-value { color: #00d4ff; }
.wo-stat.completed .wo-value { color: #00ff88; }

.wo-label {
  font-size: 12px;
  color: #888;
}

.wo-chart {
  display: flex;
  gap: 15px;
  margin-bottom: 15px;
}

.pie-chart {
  width: 80px;
  height: 80px;
```

```
  position: relative;
}

.pie-slice {
  transform: rotate(-90deg);
  transform-origin: center;
}

.pie-center {
  position: absolute;
  top: 50%;
  left: 50%;
  transform: translate(-50%, -50%);
  text-align: center;
}

.pie-value {
  font-size: 16px;
  font-weight: bold;
  color: #00ff88;
  display: block;
}

.pie-label {
  font-size: 11px;
  color: #888;
}

.wo-legend {
  flex: 1;
  display: flex;
  flex-direction: column;
  justify-content: center;
  gap: 4px;
}

.legend-item {
  display: flex;
  align-items: center;
  gap: 6px;
  font-size: 12px;
}

.legend-dot {
  width: 8px;
  height: 8px;
  border-radius: 50%;
}

.legend-text {
  flex: 1;
  color: #aaa;
}

.legend-count {
  color: #fff;
}

/* 最近工单 */
.order-list {
  display: flex;
  flex-direction: column;
  gap: 8px;
}
```

```
.order-item {
  padding: 8px;
  background: rgba(0, 50, 100, 0.2);
  border-radius: 6px;
}

.order-header {
  display: flex;
  align-items: center;
  gap: 8px;
  margin-bottom: 5px;
}

.order-priority {
  font-size: 11px;
  padding: 2px 6px;
  border-radius: 3px;
}

.order-priority.紧急 { background: #ff6b6b; color: #fff; }
.order-priority.高 { background: #ff9966; color: #fff; }
.order-priority.中 { background: #ffaa00; color: #000; }
.order-priority.低 { background: #00d4ff; color: #000; }

.order-id {
  font-size: 12px;
  color: #666;
}

.order-title {
  font-size: 13px;
  color: #fff;
  margin-bottom: 5px;
}

.order-meta {
  display: flex;
  justify-content: space-between;
  font-size: 12px;
}

.order-meta span:first-child {
  color: #888;
}

.order-status {
  padding: 2px 6px;
  border-radius: 3px;
}

.order-status.待处理 { background: rgba(255, 170, 0, 0.2); color: #ffaa00; }
.order-status.处理中 { background: rgba(0, 200, 255, 0.2); color: #00d4ff; }
.order-status.已完成 { background: rgba(0, 255, 136, 0.2); color: #00ff88; }
.order-status.已关闭 { background: rgba(255, 255, 255, 0.1); color: #888; }
</style>
```

6. 模块：参考与辅助层

参考与辅助层包括早期独立脚本版本和后端遥测桥接服务，用于保留演进痕迹并支撑外部设备数据接入。

6.1 文件：src/js/cesium.js

- 文件说明：早期 Cesium 独立脚本版本，保留地图能力演进参考。
- 行数统计：544 行

```
import * as Cesium from 'cesium'
import 'cesium/Build/Cesium/Widgets/widgets.css'

// 设置 Cesium Ion 访问令牌
Cesium.Ion.defaultAccessToken =
'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJqdGkiOiI1MGE2NjE5OC05YmU5LTRiMTctODYxOC1hZWE0YTU0NDJmM2UiLCJpZCI6Mzg5Mzg5LCJpYXQiOjE3NzA3NzE2MjR9.XlfsTR
YLQkmlFzS3Z-rGNLnchNdPlNqZUfzdX4SHtWU'

// 全局变量
let viewer
let beaconPoints = []  // 存储所有信标点位
let beaconPopups = []  // 存储每个信标点的弹窗元素
let selectedBeacon = null  // 当前选中的信标

// XX火车站坐标（地点已脱敏）
const LIUZHOU_STATION = {
  lon: 109.38871,  // 经度
  lat: 24.30755,   // 纬度
  height: 500
}

// 初始化 Cesium
export async function initCesium() {
  try {
    if (window.updateLoadingStatus) {
      window.updateLoadingStatus('正在初始化 Cesium...')
    }
    console.log('开始初始化 Cesium...')
    console.log('目标位置: XX火车站', LIUZHOU_STATION)

    // 创建基础 Viewer 配置
    const viewerOptions = {
      animation: false,
      timeline: false,
      baseLayerPicker: false,
      geocoder: false,
      homeButton: true,
      sceneModePicker: true,
      navigationHelpButton: false,
      fullscreenButton: true,
      infoBox: false,
      selectionIndicator: false
      // 不设置地形，使用默认的椭球体
    }

    // 创建 Viewer
    viewer = new Cesium.Viewer('cesiumContainer', viewerOptions)

    // 启用 Cesium World Terrain 3D 地形
    viewer.terrainProvider = await Cesium.CesiumTerrainProvider.fromIonAssetId(1, {
      requestVertexNormals: true,
      requestWaterMask: true
    })

    console.log('已启用 Cesium World Terrain 3D 地形')

    // 添加 OSM Buildings 3D 建筑图层
    try {
      if (window.updateLoadingStatus) {
        window.updateLoadingStatus('正在加载 3D 建筑图层...')
      }

      const osmBuildings = await Cesium.createOsmBuildings()
      viewer.scene.primitives.add(osmBuildings)
```

```
      console.log('✅ 已添加 OSM Buildings 3D 建筑图层')
    } catch (error) {
      console.warn('⚠️ OSM Buildings 加载失败:', error)
      // 建筑图层加载失败不影响其他功能
    }

    console.log('Cesium Viewer 创建成功')
    if (window.updateLoadingStatus) {
      window.updateLoadingStatus('Viewer 创建成功，正在添加地图图层...')
    }

    // 开启大气效果
    viewer.scene.skyAtmosphere.show = true
    viewer.scene.skyAtmosphere.hueShift = 0.1
    viewer.scene.skyAtmosphere.saturationShift = 0.1
    viewer.scene.skyAtmosphere.brightnessShift = 0.1

    // 开启雾效
    viewer.scene.fog.enabled = true
    viewer.scene.fog.density = 0.0001

    // 启用地形照明 - 增强地形立体感
    viewer.scene.globe.enableLighting = true

    // 增强地形夸张度 - 使山地地形更明显
    viewer.terrainExaggeration = 3.0  // 地形夸张倍数，默认为1.0
    console.log('地形夸张度设置为: 3.0倍')

    // 启用基于地形的遮挡检测
    viewer.scene.globe.depthTestAgainstTerrain = true

    // 设置地面透明度（可选，设置为1表示不透明）
    viewer.scene.globe.alpha = 1.0

    // 启用动态大气光影
    viewer.scene.globe.atmosphereShift = 0.05

    console.log('Cesium 初始化成功')
    if (window.updateLoadingStatus) {
      window.updateLoadingStatus('初始化完成! ');
    }
    return viewer
  } catch (error) {
    console.error('Cesium 初始化失败:', error)
    if (window.updateLoadingStatus) {
      window.updateLoadingStatus('初始化失败: ' + error.message);
    }
    throw error
  }
}

// 相机飞入到 XX火车站
export function flyToLiuZhou() {
  if (!viewer) return

  console.log('相机飞入到 XX火车站...')

  // 使用 flyTo 实现平滑的飞入效果
  viewer.camera.flyTo({
    destination: Cesium.Cartesian3.fromDegrees(
      LIUZHOU_STATION.lon,
      LIUZHOU_STATION.lat,
      LIUZHOU_STATION.height  // 飞行高度 500米
```

```
    ),
    orientation: {
      heading: Cesium.Math.toRadians(0),      // 朝向正北
      pitch: Cesium.Math.toRadians(-45),      // 俯视角度 -45度
      roll: 0.0
    },
    duration: 3.0,  // 飞行时长 3秒
    easingFunction: Cesium.EasingFunction.CUBIC_IN_OUT  // 缓动函数
  })
}

// 复位视角
window.resetView = function() {
  if (!viewer) return

  viewer.camera.flyTo({
    destination: Cesium.Cartesian3.fromDegrees(
      LIUZHOU_STATION.lon,
      LIUZHOU_STATION.lat,
      LIUZHOU_STATION.height
    ),
    orientation: {
      heading: Cesium.Math.toRadians(0),
      pitch: Cesium.Math.toRadians(-45),
      roll: 0.0
    },
    duration: 2.0
  })
}

// 更新位置信息显示
function updatePositionDisplay() {
  const lonEl = document.getElementById('longitude')
  const latEl = document.getElementById('latitude')
  const altEl = document.getElementById('altitude')

  if (lonEl && latEl && altEl) {
    const cameraPosition = viewer.camera.positionCartographic
    const lon = Cesium.Math.toDegrees(cameraPosition.longitude).toFixed(4)
    const lat = Cesium.Math.toDegrees(cameraPosition.latitude).toFixed(4)
    const alt = (cameraPosition.height * 1000).toFixed(0)

    lonEl.textContent = lon + '°E'
    latEl.textContent = lat + '°N'
    altEl.textContent = alt + 'm'
  }

  // 更新坐标显示
  const coordinatesEl = document.getElementById('coordinates')
  if (coordinatesEl) {
    const carto = viewer.camera.positionCartographic
    const lon = Cesium.Math.toDegrees(carto.longitude).toFixed(4)
    const lat = Cesium.Math.toDegrees(carto.latitude).toFixed(4)
    coordinatesEl.textContent = `${lon}, ${lat}`
  }
}

// 隐藏弹窗函数
window.hidePopup = function(beaconId) {
  if (beaconPopups[beaconId]) {
    beaconPopups[beaconId].style.display = 'none'
  }
  selectedBeacon = null
}
```

```
// 设置交互事件
function setupInteractions() {
  // 添加点击事件监听器
  const handler = new Cesium.ScreenSpaceEventHandler(viewer.scene.canvas)

  handler.setInputAction(function(click) {
    // 拾取场景中的对象
    const pickedObject = viewer.scene.pick(click.position, 0, 0)

    if (Cesium.defined(pickedObject)) {
      // 检查是否点击了信标
      for (let i = 0; i < beaconPoints.length; i++) {
        const beacon = beaconPoints[i]
        if (pickedObject.id === beacon.cylinder.id ||
            pickedObject.id === beacon.point.id ||
            pickedObject.id === beacon.wave.id) {
          // 切换弹窗显示
          const popup = beaconPopups[i]
          if (selectedBeacon === i) {
            // 如果已选中，则隐藏
            popup.style.display = 'none'
            selectedBeacon = null
          } else {
            // 隐藏其他弹窗
            beaconPopups.forEach((p, idx) => {
              if (idx !== i) p.style.display = 'none'
            })
            // 显示当前弹窗
            popup.style.display = 'block'
            selectedBeacon = i
          }
          break
        }
      }
    } else {
      // 点击空白处，隐藏所有弹窗
      beaconPopups.forEach(p => p.style.display = 'none')
      selectedBeacon = null
    }
  }, Cesium.ScreenSpaceEventType.LEFT_CLICK)

  // 监听场景渲染事件，更新弹窗位置和相机信息
  viewer.scene.postRender.addEventListener(() => {
    updatePositionDisplay()
    updatePopupPositions()
  })
}

// 更新弹窗位置，让它们跟随信标移动
function updatePopupPositions() {
  beaconPoints.forEach((beacon, index) => {
    const popup = beaconPopups[index]
    if (!popup || popup.style.display === 'none') {
      return  // 弹窗不存在或隐藏，跳过
    }

    // 转换3D坐标到屏幕坐标
    const position = Cesium.Cartesian3.fromDegrees(beacon.lon, beacon.lat, beacon.basePosition.height)
    const windowCoord = Cesium.SceneTransforms.wgs84ToWindowCoordinates(
      viewer.scene,
      position
    )
```

```
    if (Cesium.defined(windowCoord)) {
      const x = windowCoord.x - popup.offsetWidth / 2
      const y = windowCoord.y - popup.offsetHeight - 50  // 向上偏移，避免遮挡信标
      popup.style.transform = `translate3d(${Math.round(x)}px, ${Math.round(y)}px, 0)`
    }
  })
}

// 更新仪表盘数据
function updateDashboardData() {
  const lastUpdate = document.getElementById('lastUpdate')
  if (lastUpdate) {
    const now = new Date()
    lastUpdate.textContent = now.toLocaleString('zh-CN')
  }
}

// 创建随机发光柱和光波动画
function createBeaconPoints() {
  // 在目标区域周边随机生成5-8个点位
  const pointCount = Math.floor(Math.random() * 4) + 5  // 5-8个点

  for (let i = 0; i < pointCount; i++) {
    // 在目标站点周围随机分布
    const lon = LIUZHOU_STATION.lon + (Math.random() - 0.5) * 0.1  // ±0.05度
    const lat = LIUZHOU_STATION.lat + (Math.random() - 0.5) * 0.1  // ±0.05度
    const height = 200 + Math.random() * 300  // 200-500米高度

    const position = Cesium.Cartesian3.fromDegrees(lon, lat, height)

    // 生成随机事件提醒信息
    const eventTypes = ['设备正常', '温度异常', '维护中', '离线', '电压异常']
    const eventMessages = [
      '信号灯运行正常，所有参数在范围内',
      '温度超过阈值，当前45°C，请检查设备',
      '设备正在维护，预计2小时后恢复',
      '设备离线，最后检查时间：10分钟前',
      '电压异常，当前220V，需要检查线路'
    ]
    const randomEventIndex = Math.floor(Math.random() * eventTypes.length)
    const eventType = eventTypes[randomEventIndex]
    const eventMessage = eventMessages[randomEventIndex]
    const eventTime = new Date().toLocaleString('zh-CN')

    // 1. 创建发光柱（圆柱体）
    const cylinder = viewer.entities.add({
      position: position,
      cylinder: {
        length: height,
        topRadius: 2,
        bottomRadius: 2,
        material: Cesium.Color.fromCssColorString('#00d4ff').withAlpha(0.6),
        outline: true,
        outlineColor: Cesium.Color.CYAN,
        outlineWidth: 2
      }
    })

    // 2. 创建顶部发光点
    const point = viewer.entities.add({
      position: position,
      point: {
        pixelSize: 15,
        color: Cesium.Color.CYAN.withAlpha(0.8),
```

```
      outlineColor: Cesium.Color.WHITE,
      outlineWidth: 2
    }
  })

  // 3. 创建光波动画（圆形扩散）- 定位在地面
  const groundPosition = Cesium.Cartesian3.fromDegrees(lon, lat, 0)  // 地面位置

  const wave = viewer.entities.add({
    position: groundPosition,  // 放置在地面而不是顶部
    ellipse: {
      semiMinorAxis: 10,  // 初始半径较小
      semiMajorAxis: 10,
      height: 2,  // 稍微离地一点，避免z-fighting
      material: Cesium.Color.fromCssColorString('#00d4ff').withAlpha(0.6),  // 初始更不透明
      outline: true,
      outlineColor: Cesium.Color.CYAN.withAlpha(0.8),
      outlineWidth: 3,
      rotation: Cesium.Math.toRadians(Math.random() * 360)
    }
  })

  // 保存到数组中，用于动画更新
  beaconPoints.push({
    cylinder: cylinder,
    point: point,
    wave: wave,
    basePosition: { lon, lat, height },
    waveRadius: 10,  // 初始半径
    maxRadius: 200,  // 最大扩散半径
    waveSpeed: 20 + Math.random() * 30,  // 扩散速度（米/秒）
    waveAlpha: 0.6,  // 初始透明度
    waveStartTime: Date.now() + Math.random() * 2000,  // 随机延迟启动
    id: i,  // 信标ID
    lon: lon,  // 保存坐标用于弹窗
    lat: lat,
    eventType: eventType,  // 事件类型
    eventMessage: eventMessage  // 事件消息
  })

  // 创建事件提醒弹窗
  const popupDiv = document.createElement('div')
  popupDiv.className = 'beacon-popup'
  popupDiv.innerHTML = `
    <div class="popup-content">
      <div class="popup-title">🚨 信号灯 ${i + 1} - ${eventType}</div>
      <div class="popup-time">⏰ ${eventTime}</div>
      <div class="popup-message">${eventMessage}</div>
      <div class="popup-coord">📍 坐标: ${lon.toFixed(4)}, ${lat.toFixed(4)}</div>
      <div class="popup-close" onclick="event.stopPropagation(); hidePopup(${i})">✕</div>
    </div>
  `
  popupDiv.style.cssText = `
    position: absolute;
    left: 0;
    top: 0;
    display: none;
    z-index: 1000;
    pointer-events: auto;
  `

  viewer.container.appendChild(popupDiv)
  beaconPopups.push(popupDiv)
```

```
    console.log(`创建信标点 ${i + 1}: [${lon.toFixed(4)}, ${lat.toFixed(4)}, ${height.toFixed(0)}m]`)
  }

  // 添加CSS样式到页面
  const style = document.createElement('style')
  style.textContent = `
    .beacon-popup {
      background: rgba(0, 20, 40, 0.95);
      border: 2px solid rgba(0, 200, 255, 0.6);
      border-radius: 8px;
      padding: 15px;
      min-width: 280px;
      box-shadow: 0 4px 20px rgba(0, 0, 0, 0.5);
      backdrop-filter: blur(10px);
      font-family: 'Microsoft YaHei', Arial, sans-serif;
    }
    .popup-content {
      color: #fff;
    }
    .popup-title {
      font-size: 16px;
      font-weight: bold;
      color: #00d4ff;
      margin-bottom: 10px;
      padding-bottom: 8px;
      border-bottom: 1px solid rgba(0, 200, 255, 0.3);
    }
    .popup-time {
      font-size: 12px;
      color: #aaa;
      margin-bottom: 8px;
    }
    .popup-message {
      font-size: 13px;
      line-height: 1.5;
      margin-bottom: 10px;
    }
    .popup-coord {
      font-size: 11px;
      color: #888;
      font-family: monospace;
    }
    .popup-close {
      position: absolute;
      top: 8px;
      right: 8px;
      cursor: pointer;
      font-size: 18px;
      color: #ff6666;
      background: rgba(255, 255, 255, 0.1);
      border-radius: 4px;
      width: 24px;
      height: 24px;
      display: flex;
      align-items: center;
      justify-content: center;
    }
    .popup-close:hover {
      background: rgba(255, 255, 255, 0.2);
      color: #ff3333;
    }
  `
  document.head.appendChild(style)
}
```

```
// 更新光波动画 - 像地震波一样从中心向外发散
function updateWaveAnimation() {
  const currentTime = Date.now()

  beaconPoints.forEach((beacon, index) => {
    // 检查是否到了该光波的启动时间
    if (currentTime < beacon.waveStartTime) {
      return  // 还没到启动时间，跳过
    }

    // 计算从启动开始经过的时间（秒）
    const elapsedTime = (currentTime - beacon.waveStartTime) * 0.001

    // 计算当前半径：从10米开始向外扩散
    const currentRadius = beacon.waveRadius + elapsedTime * beacon.waveSpeed

    // 如果超过最大半径，重置（循环效果）
    if (currentRadius > beacon.maxRadius) {
      beacon.waveStartTime = currentTime  // 重置启动时间
      beacon.wave.ellipse.semiMinorAxis = beacon.waveRadius
      beacon.wave.ellipse.semiMajorAxis = beacon.waveRadius
      beacon.wave.ellipse.material = Cesium.Color.fromCssColorString('#00d4ff').withAlpha(beacon.waveAlpha)
    } else {
      // 更新光波半径 - 从中心向外扩散
      beacon.wave.ellipse.semiMinorAxis = currentRadius
      beacon.wave.ellipse.semiMajorAxis = currentRadius

      // 透明度随半径增大而减小（扩散越远越透明）
      const progress = (currentRadius - beacon.waveRadius) / (beacon.maxRadius - beacon.waveRadius)
      const newAlpha = beacon.waveAlpha * (1 - progress * 0.9)  // 从0.6渐变到0.06
      beacon.wave.ellipse.material = Cesium.Color.fromCssColorString('#00d4ff').withAlpha(newAlpha)
    }

    // 顶部发光点脉冲效果（独立于光波）
    const pulseTime = currentTime * 0.003
    const pulseSize = 12 + Math.sin(pulseTime + index * 2) * 5  // 7-17像素
    beacon.point.point.pixelSize = pulseSize
  })
}

// 主初始化函数
export async function init() {
  try {
    console.log('=== Cesium 大地图初始化开始 ===')

    // 初始化 Cesium
    await initCesium()

    // 创建随机发光柱和光波
    createBeaconPoints()

    // 相机飞入到 XX火车站
    if (window.updateLoadingStatus) {
      window.updateLoadingStatus('正在飞向XX火车站...')
    }

    setTimeout(() => {
      flyToLiuZhou()
    }, 500)

    // 设置交互
    setupInteractions()
```

```
    // 启动光波动画循环
    viewer.clock.onTick.addEventListener(updateWaveAnimation)

    // 定时更新仪表盘数据
    setInterval(updateDashboardData, 1000)

    // 隐藏加载动画
    setTimeout(() => {
      const loading = document.getElementById('loading')
      if (loading) loading.classList.add('hidden')
      console.log('=== Cesium 大地图初始化完成 ===')
    }, 2000)

  } catch (error) {
    console.error('初始化失败:', error)
    if (window.updateLoadingStatus) {
      window.updateLoadingStatus('初始化失败: ' + error.message);
    }
    const loading = document.getElementById('loading')
    if (loading) {
      loading.innerHTML = `
        <div style="color: #ff6666;">
          <div style="font-size: 24px; margin-bottom: 20px;">❌ 加载失败</div>
          <div style="font-size: 14px;">${error.message}</div>
          <div style="font-size: 12px; margin-top: 20px;">请打开浏览器控制台查看详细错误信息</div>
        </div>
      `
    }
  }
}

// 启动应用
init()
```

6.2 文件：src/js/threejs.js

- 文件说明：早期 Three.js 独立脚本版本，保留三维孪生能力演进参考。
- 行数统计：726 行

```
import * as THREE from 'three'
import { OrbitControls } from 'three/examples/jsm/controls/OrbitControls.js'
import { GLTFLoader } from 'three/examples/jsm/loaders/GLTFLoader.js'
import { CSS2DRenderer } from 'three/examples/jsm/renderers/CSS2DRenderer.js'

// 全局变量
let scene, camera, renderer, controls, labelRenderer
let signals = []
let train = null
let isTrainRunning = false
let autoRotate = false
let clock = new THREE.Clock()
let startTime = Date.now()
let mixer = null // 动画混合器
let gltfLoader = null
let modelLabels = []  // 存储所有模型标签

// 进度日志节流：减少“加载进度”刷屏
function createProgressLogger(label, step = 25) {
  let lastBucket = -1
  return (progress) => {
    if (!progress || !progress.total) return
    const percent = Math.floor((progress.loaded / progress.total) * 100)
    const bucket = Math.floor(percent / step) * step
    if (bucket === lastBucket) return
    lastBucket = bucket
    console.log(`${label} 加载进度: ${percent}%`)
```

```
  }
}

// 初始化 Three.js
function init() {
  // 创建场景
  scene = new THREE.Scene()
  scene.background = new THREE.Color(0xf0f0f0) // 改为白色背景
  scene.fog = new THREE.Fog(0xf0f0f0, 100, 800) // 雾效也改为白色

  // 创建相机
  camera = new THREE.PerspectiveCamera(
    60,
    window.innerWidth / window.innerHeight,
    0.1,
    2000
  )
  camera.position.set(80, 60, 80)

  // 创建渲染器
  renderer = new THREE.WebGLRenderer({ antialias: true })
  renderer.setSize(window.innerWidth, window.innerHeight)
  renderer.setPixelRatio(window.devicePixelRatio)
  renderer.shadowMap.enabled = true
  renderer.shadowMap.type = THREE.PCFSoftShadowMap
  document.getElementById('threeContainer').appendChild(renderer.domElement)

  // 创建CSS2D渲染器用于HTML标签
  labelRenderer = new CSS2DRenderer()
  labelRenderer.setSize(window.innerWidth, window.innerHeight)
  labelRenderer.domElement.style.position = 'absolute'
  labelRenderer.domElement.style.top = '0'
  labelRenderer.domElement.style.pointerEvents = 'none'
  document.body.appendChild(labelRenderer.domElement)

  // 创建控制器
  controls = new OrbitControls(camera, renderer.domElement)
  controls.enableDamping = true
  controls.dampingFactor = 0.05
  controls.maxPolarAngle = Math.PI / 2
  controls.minDistance = 20
  controls.maxDistance = 200

  // 添加光照
  setupLights()

  // 创建场景
  createGround()
  createMountains()
  createRailway()
  createSignalLights()
  createTrain()  // 恢复火车头
  // createStation()  // 已禁用 - 移除火车站
  createTrees()

  // 事件监听
  window.addEventListener('resize', onWindowResize)

  // 隐藏加载动画
  setTimeout(() => {
    const loading = document.getElementById('loading')
    if (loading) loading.classList.add('hidden')
  }, 1000)
```

```
  // 开始动画
  animate()

  // 更新 UI
  updateUI()
  setInterval(updateUI, 1000)

  console.log('Three.js 小地图初始化完成')
}

// 设置光照
function setupLights() {
  // 环境光 - 增强亮度
  const ambientLight = new THREE.AmbientLight(0xffffff, 0.8)
  scene.add(ambientLight)

  // 主光源 - 增强亮度
  const mainLight = new THREE.DirectionalLight(0xffffff, 1.5)
  mainLight.position.set(50, 100, 50)
  mainLight.castShadow = true
  mainLight.shadow.camera.left = -100
  mainLight.shadow.camera.right = 100
  mainLight.shadow.camera.top = 100
  mainLight.shadow.camera.bottom = -100
  mainLight.shadow.mapSize.width = 2048
  mainLight.shadow.mapSize.height = 2048
  scene.add(mainLight)

  // 补光 - 改为暖色
  const fillLight = new THREE.DirectionalLight(0xffeedd, 0.5)
  fillLight.position.set(-50, 50, -50)
  scene.add(fillLight)

  // 半球光 - 提供更自然的照明
  const hemiLight = new THREE.HemisphereLight(0xffffff, 0x444444, 0.6)
  hemiLight.position.set(0, 200, 0)
  scene.add(hemiLight)
}

// 创建地面
function createGround() {
  const groundGeometry = new THREE.PlaneGeometry(500, 500, 50, 50)
  const groundMaterial = new THREE.MeshStandardMaterial({
    color: 0x7a8a6a, // 浅绿色地面
    roughness: 0.9,
    metalness: 0.1
  })

  // 添加地形起伏
  const vertices = groundGeometry.attributes.position.array
  for (let i = 0; i < vertices.length; i += 3) {
    const x = vertices[i]
    const z = vertices[i + 2]
    vertices[i + 1] = Math.sin(x * 0.05) * Math.cos(z * 0.05) * 5
  }
  groundGeometry.computeVertexNormals()

  const ground = new THREE.Mesh(groundGeometry, groundMaterial)
  ground.rotation.x = -Math.PI / 2
  ground.receiveShadow = true
  scene.add(ground)
}

// 创建山脉
```

```
function createMountains() {
  const mountainGeometry = new THREE.ConeGeometry(30, 60, 8)
  const mountainMaterial = new THREE.MeshStandardMaterial({
    color: 0x8a9a8a, // 浅灰色山脉
    roughness: 0.95,
    metalness: 0.05
  })

  const positions = [
    { x: -80, z: -80 },
    { x: -100, z: 50 },
    { x: 80, z: -100 },
    { x: 100, z: 60 },
    { x: -60, z: 120 }
  ]

  positions.forEach(pos => {
    const mountain = new THREE.Mesh(mountainGeometry, mountainMaterial)
    mountain.position.set(pos.x, 30, pos.z)
    mountain.scale.y = 1 + Math.random() * 0.5
    mountain.castShadow = true
    mountain.receiveShadow = true
    scene.add(mountain)
  })
}

// 创建铁道（加载 GLB 模型并拼接）
function createRailway() {
  if (!gltfLoader) {
    gltfLoader = new GLTFLoader()
  }

  // 定义铁道路径点 - 9段轨道在一直线上，首尾相接，统一旋转50度
  const railPositions = [
    { x: -80, z: -60, rotation: Math.PI / 3.6 },  // 轨道1
    { x: -60, z: -45, rotation: Math.PI / 3.6 },  // 轨道2
    { x: -40, z: -30, rotation: Math.PI / 3.6 },  // 轨道3
    { x: -20, z: -15, rotation: Math.PI / 3.6 },  // 轨道4
    { x: 0, z: 0, rotation: Math.PI / 3.6 },      // 轨道5（中心）
    { x: 20, z: 15, rotation: Math.PI / 3.6 },   // 轨道6
    { x: 40, z: 30, rotation: Math.PI / 3.6 },   // 轨道7
    { x: 60, z: 45, rotation: Math.PI / 3.6 },   // 轨道8
    { x: 80, z: 60, rotation: Math.PI / 3.6 }    // 轨道9
  ]

  // 加载并放置多个铁轨段
  railPositions.forEach((pos, index) => {
    const logProgress = createProgressLogger(`铁轨段 ${index + 1}`, 25)
    gltfLoader.load(
      '/assets/models/railway.glb',
      (gltf) => {
        const railway = gltf.scene
        railway.scale.set(12, 12, 12) // 放大12倍
        railway.position.set(pos.x, 0, pos.z) // 放在地平线上
        railway.rotation.y = pos.rotation

        // 启用阴影
        railway.traverse((child) => {
          if (child.isMesh) {
            child.castShadow = true
            child.receiveShadow = true
          }
        })
```

```
        scene.add(railway)
        console.log(`铁轨段 ${index + 1} 加载成功`)
      },
      (progress) => {
        logProgress(progress)
      },
      (error) => {
        console.error(`铁轨段 ${index + 1} 加载失败:`, error)
      }
    )
  })

  // 创建路径用于火车运动
  const curve = new THREE.CatmullRomCurve3([
    new THREE.Vector3(-60, 0.5, -40),
    new THREE.Vector3(-30, 0.5, -20),
    new THREE.Vector3(0, 0.5, 0),
    new THREE.Vector3(30, 0.5, 20),
    new THREE.Vector3(60, 0.5, 40)
  ])

  return curve
}

// 创建信号灯（加载 GLB 模型）
function createSignalLights() {
  if (!gltfLoader) {
    gltfLoader = new GLTFLoader()
  }

  const signalPositions = [
    { x: -40, z: -25 },
    { x: -10, z: -5 },
    { x: 20, z: 15 },
    { x: 40, z: 30 }
  ]

  const signalStates = ['red', 'green', 'yellow', 'green']
  const signalNames = ['进站信号', '出站信号', '区间信号1', '区间信号2']

  signalPositions.forEach((pos, index) => {
    const logProgress = createProgressLogger(`信号灯 ${signalNames[index]}`, 25)
    gltfLoader.load(
      '/assets/models/sign.glb',
      (gltf) => {
        const signGroup = gltf.scene
        signGroup.scale.set(8, 8, 8) // 放大8倍
        signGroup.position.set(pos.x, 0, pos.z) // 放在地平线上

        // 启用阴影
        signGroup.traverse((child) => {
          if (child.isMesh) {
            child.castShadow = true
            child.receiveShadow = true
          }
        })

        scene.add(signGroup)

        // 添加点光源（根据信号灯状态）
        const color = getColorByState(signalStates[index])
        const light = new THREE.PointLight(color, 3, 40) // 增强光源强度和范围
        light.position.set(pos.x, 5, pos.z)
        scene.add(light)
```

```
        signals.push({
          mesh: signGroup,
          light: light,
          state: signalStates[index],
          name: signalNames[index]
        })

        // 创建3D标签
        createModelLabel(signGroup, signalNames[index], 15, 65, 45, '109.3887, 24.3076')

        updateSignalUI()
        console.log(`信号灯 ${signalNames[index]} 加载成功`)
      },
      (progress) => {
        logProgress(progress)
      },
      (error) => {
        console.error(`信号灯 ${signalNames[index]} 加载失败:`, error)
      }
    )
  })
}

// 根据状态获取颜色
function getColorByState(state) {
  switch (state) {
    case 'red': return 0xff3333
    case 'green': return 0x33ff33
    case 'yellow': return 0xffff33
    default: return 0x666666
  }
}

// 创建列车（加载 GLB 模型）
function createTrain() {
  if (!gltfLoader) {
    gltfLoader = new GLTFLoader()
  }

  const logProgress = createProgressLogger('火车头', 25)
  gltfLoader.load(
    '/assets/models/locomotive.glb',
    (gltf) => {
      train = gltf.scene
      train.scale.set(12, 12, 12) // 放大12倍
      train.position.set(-60, 0.5, -40) // 稍微抬高，放在铁轨上
      train.rotation.y = Math.PI / 2 // 调整朝向

      // 启用阴影
      train.traverse((child) => {
        if (child.isMesh) {
          child.castShadow = true
          child.receiveShadow = true
        }
      })

      scene.add(train)

      // 如果有动画，设置动画混合器
      if (gltf.animations && gltf.animations.length > 0) {
        mixer = new THREE.AnimationMixer(train)
        const action = mixer.clipAction(gltf.animations[0])
        action.play()
```

```
      }

      // 创建3D标签
      createModelLabel(train, '火车头', -5, 75, 60, '109.3900, 24.3100')

      console.log('火车头模型加载成功')
    },
    (progress) => {
      logProgress(progress)
    },
    (error) => {
      console.error('火车头模型加载失败:', error)
      // 如果模型加载失败，使用简单的几何体作为后备
      createFallbackTrain()
    }
  )
}

// 后备火车（如果 GLB 加载失败）
function createFallbackTrain() {
  const trainGroup = new THREE.Group()

  // 车身
  const bodyGeometry = new THREE.BoxGeometry(8, 3, 3)
  const bodyMaterial = new THREE.MeshStandardMaterial({ color: 0x2244aa })
  const body = new THREE.Mesh(bodyGeometry, bodyMaterial)
  body.position.y = 2
  body.castShadow = true
  trainGroup.add(body)

  // 车头
  const headGeometry = new THREE.BoxGeometry(2, 2.5, 2.8)
  const headMaterial = new THREE.MeshStandardMaterial({ color: 0x3355bb })
  const head = new THREE.Mesh(headGeometry, headMaterial)
  head.position.set(5, 2, 0)
  head.castShadow = true
  trainGroup.add(head)

  // 车窗
  const windowGeometry = new THREE.BoxGeometry(0.1, 1, 2)
  const windowMaterial = new THREE.MeshStandardMaterial({
    color: 0x88ccff,
    emissive: 0x4488cc,
    emissiveIntensity: 0.3
  })
  const trainWindow = new THREE.Mesh(windowGeometry, windowMaterial)
  trainWindow.position.set(6.1, 2.5, 0)
  trainGroup.add(trainWindow)

  // 车轮
  const wheelGeometry = new THREE.CylinderGeometry(0.6, 0.6, 0.3, 16)
  const wheelMaterial = new THREE.MeshStandardMaterial({ color: 0x222222 })
  const wheelPositions = [[-3, 0.6, 1.5], [-3, 0.6, -1.5], [3, 0.6, 1.5], [3, 0.6, -1.5]]

  wheelPositions.forEach(pos => {
    const wheel = new THREE.Mesh(wheelGeometry, wheelMaterial)
    wheel.position.set(...pos)
    wheel.rotation.x = Math.PI / 2
    wheel.castShadow = true
    trainGroup.add(wheel)
  })

  train = trainGroup
  train.position.set(-60, 0, -40)
```

```
  scene.add(train)
}

// 创建火车站（加载 GLB 模型）
function createStation() {
  if (!gltfLoader) {
    gltfLoader = new GLTFLoader()
  }

  const logProgress = createProgressLogger('火车站', 25)
  gltfLoader.load(
    '/assets/models/station.glb',
    (gltf) => {
      const station = gltf.scene
      station.scale.set(20, 20, 20) // 放大20倍
      station.position.set(30, 0, 20) // 在地平线上，与铁轨、信号灯同高
      station.rotation.y = -Math.PI / 4 // 调整朝向

      // 启用阴影
      station.traverse((child) => {
        if (child.isMesh) {
          child.castShadow = true
          child.receiveShadow = true
        }
      })

      scene.add(station)
      console.log('火车站模型加载成功')

      // 创建3D标签
      createModelLabel(station, 'XX火车站', 30, -5, 85, 'X, Y')
    },
    (progress) => {
      logProgress(progress)
    },
    (error) => {
      console.error('火车站模型加载失败:', error)
    }
  )
}

// 创建树木
function createTrees() {
  const treeCount = 30

  for (let i = 0; i < treeCount; i++) {
    const treeGroup = new THREE.Group()

    // 树干
    const trunkGeometry = new THREE.CylinderGeometry(0.3, 0.5, 3, 8)
    const trunkMaterial = new THREE.MeshStandardMaterial({ color: 0x6a5a4a })
    const trunk = new THREE.Mesh(trunkGeometry, trunkMaterial)
    trunk.position.y = 1.5
    trunk.castShadow = true
    treeGroup.add(trunk)

    // 树冠
    const leavesGeometry = new THREE.ConeGeometry(2, 4, 8)
    const leavesMaterial = new THREE.MeshStandardMaterial({ color: 0x4a8a4a })
    const leaves = new THREE.Mesh(leavesGeometry, leavesMaterial)
    leaves.position.y = 5
    leaves.castShadow = true
    treeGroup.add(leaves)
```

```
    // 随机位置
    const x = (Math.random() - 0.5) * 200
    const z = (Math.random() - 0.5) * 200
    treeGroup.position.set(x, 0, z)

    scene.add(treeGroup)
  }
}

// 创建3D模型的HTML标签
function createModelLabel(model, name, temperature, humidity, gpsLon, gpsLat) {
  // 创建标签容器div
  const div = document.createElement('div')
  div.className = 'model-label'

  // 根据温度确定颜色
  let tempClass = 'temp-low'
  if (temperature < 0) tempClass = 'temp-low'
  else if (temperature < 20) tempClass = 'temp-medium'
  else tempClass = 'temp-high'

  // 计算温度进度条宽度
  const tempPercent = Math.min(Math.abs(temperature) / 40 * 100, 100)

  div.innerHTML = `
    <div class="label-title">${name}</div>
    <div class="label-row">
      <span>🌡️ 温度:</span>
      <span class="label-value">${temperature}°C</span>
    </div>
    <div class="label-row">
      <span>💧 湿度:</span>
      <span class="label-value">${humidity}%</span>
    </div>
    <div class="label-row">
      <span>📍 GPS:</span>
      <span class="label-value">${gpsLon}, ${gpsLat}</span>
    </div>
    <div class="label-row">
      <span style="flex: 1; margin-left: 10px;">
        <span>温度:</span>
        <span style="flex: 1; margin-left: auto;">
          <div class="progress-bg">
            <div class="temp-bar ${tempClass}" style="width: ${tempPercent}%"></div>
          </div>
        </span>
      </span>
    </div>
  `

  // 添加到场景和标签渲染器
  const label = new THREE.CSS2DObject(div)
  label.position.set(0, 0, 0)
  model.add(label)
  modelLabels.push({ object: model, label: label })
}

// 切换信号灯
window.toggleSignals = function() {
  const states = ['red', 'green', 'yellow']
  signals.forEach(signal => {
    const currentStateIndex = states.indexOf(signal.state)
    signal.state = states[(currentStateIndex + 1) % states.length]
    const color = getColorByState(signal.state)
```

```
    // 更新光源颜色
    signal.light.color.setHex(color)

    // 尝试更新模型中 mesh 的颜色（如果模型支持）
    if (signal.mesh) {
      signal.mesh.traverse((child) => {
        if (child.isMesh) {
          // 尝试识别信号灯部分（通常名称包含 'light' 或 'signal'）
          if (child.material) {
            // 如果材质有 emissive 属性，修改它
            if (child.material.emissive) {
              child.material.emissive.setHex(color)
            }
            // 如果材质有 color 属性，也修改它
            if (child.material.color) {
              child.material.color.setHex(color)
            }
          }
        }
      })
    }
  })
  updateSignalUI()
}

// 更新信号灯 UI
function updateSignalUI() {
  const signalList = document.getElementById('signalList')
  if (!signalList) return

  const stateText = {
    'red': '禁止通行',
    'green': '允许通行',
    'yellow': '减速通行'
  }

  signalList.innerHTML = signals.map(signal => `
    <div class="signal-item">
      <div class="signal-light signal-${signal.state}"></div>
      <div class="signal-info">
        <div class="signal-name">${signal.name}</div>
        <div class="signal-status">${stateText[signal.state]}</div>
      </div>
    </div>
  `).join('')

  const countEl = document.getElementById('signalCount')
  if (countEl) countEl.textContent = signals.length
}

// 列车动画
window.trainAnimation = function() {
  isTrainRunning = !isTrainRunning
}

// 自动旋转
window.toggleRotation = function() {
  autoRotate = !autoRotate
}

// 复位视角
window.resetView = function() {
  camera.position.set(80, 60, 80)
```

```
  controls.target.set(0, 0, 0)
  controls.update()
}

// 切换摄像头
window.switchCamera = function(cameraId) {
  // 移除所有tab的active类
  const tabs = document.querySelectorAll('.camera-tab')
  tabs.forEach(tab => tab.classList.remove('active'))

  // 隐藏所有摄像头内容
  const contents = document.querySelectorAll('.camera-content')
  contents.forEach(content => content.classList.remove('active'))

  // 激活选中的tab和内容
  const selectedTab = document.querySelector(`.camera-tab:nth-child(${cameraId})`)
  const selectedContent = document.getElementById(`camera${cameraId}`)

  if (selectedTab) selectedTab.classList.add('active')
  if (selectedContent) selectedContent.classList.add('active')

  console.log(`切换到监控${cameraId}`)
}

// 窗口大小调整
function onWindowResize() {
  camera.aspect = window.innerWidth / window.innerHeight
  camera.updateProjectionMatrix()
  renderer.setSize(window.innerWidth, window.innerHeight)
  labelRenderer.setSize(window.innerWidth, window.innerHeight)
}

// 更新 UI
function updateUI() {
  // 更新系统状态
  const lastUpdate = document.getElementById('lastUpdate')
  if (lastUpdate) {
    lastUpdate.textContent = new Date().toLocaleString('zh-CN')
  }

  // 更新运行时间
  const uptime = document.getElementById('uptime')
  if (uptime) {
    const seconds = Math.floor((Date.now() - startTime) / 1000)
    uptime.textContent = `${seconds}s`
  }

  // 更新相机信息
  const camPos = document.getElementById('cameraPos')
  if (camPos) {
    camPos.textContent = `${camera.position.x.toFixed(0)}, ${camera.position.y.toFixed(0)}, ${camera.position.z.toFixed(0)}`
  }

  const camTarget = document.getElementById('cameraTarget')
  if (camTarget) {
    camTarget.textContent = `${controls.target.x.toFixed(0)}, ${controls.target.y.toFixed(0)}, ${controls.target.z.toFixed(0)}`
  }

  // 更新性能信息
  const fps = document.getElementById('fps')
  if (fps) {
    fps.textContent = '60 FPS'
  }
```

```
  const triangles = document.getElementById('triangles')
  if (triangles) {
    triangles.textContent = renderer.info.render.triangles.toLocaleString()
  }

  const drawCalls = document.getElementById('drawCalls')
  if (drawCalls) {
    drawCalls.textContent = renderer.info.render.calls
  }
}

// 动画循环
function animate() {
  requestAnimationFrame(animate)

  const delta = clock.getDelta()

  // 更新控制器
  controls.update()

  // 自动旋转
  if (autoRotate) {
    controls.autoRotate = true
    controls.autoRotateSpeed = 2.0
  } else {
    controls.autoRotate = false
  }

  // 列车动画
  if (isTrainRunning && train) {
    const time = clock.getElapsedTime() * 0.5
    const path = new THREE.CatmullRomCurve3([
      new THREE.Vector3(-60, 0.5, -40),
      new THREE.Vector3(-30, 0.5, -20),
      new THREE.Vector3(0, 0.5, 0),
      new THREE.Vector3(30, 0.5, 20),
      new THREE.Vector3(60, 0.5, 40)
    ])
    const position = path.getPoint((time % 1))
    train.position.copy(position)

    // 计算方向
    const nextPoint = path.getPoint(((time + 0.01) % 1))
    train.lookAt(nextPoint)
  }

  // 更新动画混合器（如果火车模型有动画）
  if (mixer) {
    mixer.update(delta)
  }

  // 渲染
  renderer.render(scene, camera)
  labelRenderer.render(scene, camera)  // 渲染HTML标签
}

// 启动应用
init()
```

6.3 文件：server/telemetry-bridge.js

- 文件说明：Node.js 遥测桥接服务，提供 HTTP 接收与 WebSocket 广播。
- 行数统计：189 行

```
import http from 'node:http'
import { WebSocketServer } from 'ws'
```

```
const PORT = Number(process.env.TELEMETRY_PORT || 8080)
const WS_PATH = process.env.TELEMETRY_WS_PATH || '/ws'
const UPLOAD_PATH = process.env.TELEMETRY_UPLOAD_PATH || '/upload'
const MAX_BODY_BYTES = Number(process.env.TELEMETRY_MAX_BODY_BYTES || 32 * 1024)

const now = () => new Date().toISOString()
const log = (msg) => console.log(`${now()} [telemetry] ${msg}`)

const writeCommonHeaders = (res) => {
  res.setHeader('Access-Control-Allow-Origin', '*')
  res.setHeader('Access-Control-Allow-Methods', 'POST,OPTIONS,GET')
  res.setHeader('Access-Control-Allow-Headers', 'Content-Type,*')
  res.setHeader('Cache-Control', 'no-store')
}

const safeJsonParse = (text) => {
  try {
    return { ok: true, value: JSON.parse(text) }
  } catch (e) {
    return { ok: false, error: e instanceof Error ? e.message : String(e) }
  }
}

const normalizeTelemetry = (payload) => {
  const distanceCm = payload?.distance_cm
  const distanceCmNum = Number.isFinite(Number(distanceCm)) ? Number(distanceCm) : null
  const distanceM = distanceCmNum == null ? null : distanceCmNum / 100

  const waterActive = payload?.water_active
  const waterActiveNum = Number.isFinite(Number(waterActive)) ? Number(waterActive) : null

  return {
    type: payload?.type || 'telemetry',
    device: payload?.device || 'unknown',
    device_id: payload?.device_id || 'unknown',
    ts_ms: Number.isFinite(Number(payload?.ts_ms)) ? Number(payload.ts_ms) : Date.now(),
    uptime_ms: Number.isFinite(Number(payload?.uptime_ms)) ? Number(payload.uptime_ms) : null,
    wifi_ip: typeof payload?.wifi_ip === 'string' ? payload.wifi_ip : null,
    distance_cm: distanceCmNum == null ? null : Math.round(distanceCmNum * 10) / 10,
    distance_m: distanceM == null ? null : Math.round(distanceM * 100) / 100,
    water_active: waterActiveNum == null ? null : waterActiveNum,
    raw: payload
  }
}

let lastTelemetry = null

const broadcastTelemetry = (msg, meta = {}) => {
  const encoded = JSON.stringify({ topic: 'telemetry', data: msg })
  let sent = 0
  let skipped = 0
  for (const client of wss.clients) {
    if (client.readyState === 1) {
      client.send(encoded)
      sent++
    } else {
      skipped++
    }
  }
  return { sent, skipped, clients: wss.clients.size, ...meta }
}

const server = http.createServer((req, res) => {
```

```
  if (!req.url) {
    res.statusCode = 400
    return res.end('Bad request')
  }

  const url = new URL(req.url, `http://${req.headers.host || 'localhost'}`)
  const pathname = url.pathname

  if (req.method === 'OPTIONS') {
    writeCommonHeaders(res)
    res.statusCode = 204
    return res.end()
  }

  if (req.method === 'GET' && pathname === '/telemetry/health') {
    writeCommonHeaders(res)
    res.statusCode = 200
    res.setHeader('Content-Type', 'application/json; charset=utf-8')
    return res.end(
      JSON.stringify({
        ok: true,
        port: PORT,
        ws_path: WS_PATH,
        upload_path: UPLOAD_PATH,
        clients: wss.clients.size,
        last: lastTelemetry
          ? { ts_ms: lastTelemetry.ts_ms, device: lastTelemetry.device, type: lastTelemetry.type }
          : null
      })
    )
  }

  if (req.method === 'POST' && pathname === UPLOAD_PATH) {
    const remote = req.socket?.remoteAddress || 'unknown'
    const contentType = typeof req.headers['content-type'] === 'string' ? req.headers['content-type'] : 'unknown'
    const contentLength = typeof req.headers['content-length'] === 'string' ? req.headers['content-length'] : 'unknown'
    log(`upload start: from=${remote} ct=${contentType} len=${contentLength}`)
    let body = ''
    let bodyBytes = 0

    req.setEncoding('utf8')
    req.on('data', (chunk) => {
      bodyBytes += Buffer.byteLength(chunk, 'utf8')
      if (bodyBytes > MAX_BODY_BYTES) {
        log(`upload rejected: 413 too large bytes=${bodyBytes} from=${remote}`)
        writeCommonHeaders(res)
        res.statusCode = 413
        res.setHeader('Content-Type', 'application/json; charset=utf-8')
        res.end(JSON.stringify({ ok: false, error: 'payload too large' }))
        req.destroy()
        return
      }
      body += chunk
    })

    req.on('end', () => {
      log(`upload end: bytes=${bodyBytes} from=${remote}`)
      const parsed = safeJsonParse(body || '{}')
      if (!parsed.ok) {
        log(`upload rejected: 400 invalid json from=${remote} detail=${parsed.error}`)
        writeCommonHeaders(res)
        res.statusCode = 400
        res.setHeader('Content-Type', 'application/json; charset=utf-8')
        res.end(JSON.stringify({ ok: false, error: 'invalid json', detail: parsed.error }))
```

```
        return
      }

      const msg = normalizeTelemetry(parsed.value)
      lastTelemetry = msg

      const { sent, skipped } = broadcastTelemetry(msg, { from: remote })

      log(
        `upload ok: device=${msg.device} id=${msg.device_id} type=${msg.type} distance_cm=${msg.distance_cm ?? 'null'} water=${msg.water_active
?? 'null'} bytes=${bodyBytes} ws_sent=${sent} ws_skipped=${skipped} clients=${wss.clients.size} from=${remote}`
      )

      writeCommonHeaders(res)
      res.statusCode = 200
      res.setHeader('Content-Type', 'application/json; charset=utf-8')
      res.end(JSON.stringify({ ok: true }))
    })

    req.on('error', (err) => {
      writeCommonHeaders(res)
      res.statusCode = 500
      res.setHeader('Content-Type', 'application/json; charset=utf-8')
      res.end(JSON.stringify({ ok: false, error: err?.message || 'request error' }))
    })

    return
  }

  writeCommonHeaders(res)
  res.statusCode = 404
  res.setHeader('Content-Type', 'application/json; charset=utf-8')
  res.end(JSON.stringify({ ok: false, error: 'Not Found' }))
})

const wss = new WebSocketServer({ server, path: WS_PATH })

wss.on('connection', (ws, req) => {
  log(`ws connected: ${req?.socket?.remoteAddress || 'unknown'} clients=${wss.clients.size}`)
  if (lastTelemetry) {
    ws.send(JSON.stringify({ topic: 'telemetry', data: lastTelemetry }))
  }

  ws.on('message', (data) => {
    // optional ping/pong or client subscriptions
    const text = typeof data === 'string' ? data : data.toString('utf8')
    if (text === 'ping') ws.send('pong')
  })

  ws.on('close', () => {
    log(`ws closed: clients=${wss.clients.size}`)
  })
})

server.listen(PORT, () => {
  log(`listening: http://localhost:${PORT}`)
  log(`upload: POST ${UPLOAD_PATH}`)
  log(`ws: ${WS_PATH}`)
})
```

6.4 文件：package.json

- 文件说明：项目依赖与开发命令配置。
- 行数统计：42 行

```
{
  "name": "railway_sign",
  "version": "1.0.0",
  "main": "index.js",
  "type": "module",
  "scripts": {
    "telemetry": "node server/telemetry-bridge.js",
    "dev": "vite",
    "dev:all": "concurrently -n telemetry,web -c magenta,cyan \"npm:telemetry\" \"npm:dev\"",
    "build": "vite build",
    "preview": "vite preview",
    "softdocs": "node scripts/build-soft-copyright-pdfs.mjs"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": "",
  "dependencies": {
    "cesium": "^1.137.0",
    "echarts": "^6.0.0",
    "gsap": "^3.14.2",
    "lil-gui": "^0.21.0",
    "mqtt": "^5.14.1",
    "three": "^0.182.0",
    "vue": "^3.5.28",
    "ws": "^8.19.0"
  },
  "devDependencies": {
    "@pdf-lib/fontkit": "^1.1.1",
    "@vitejs/plugin-vue": "^6.0.4",
    "concurrently": "^9.2.0",
    "dompurify": "^3.3.2",
    "jsdom": "^28.1.0",
    "marked": "^17.0.4",
    "mermaid": "^11.13.0",
    "pdf-lib": "^1.17.1",
    "typescript": "^5.9.3",
    "vite": "^7.3.1",
    "vite-plugin-cesium": "^1.2.23"
  }
}
```

7. 总行数统计说明

本次纳入整理的真实文件共 20 个，总计 14361 行；其中 `src` 目录代码量 14098 行，已超过软著材料常见的 3000 行整理需求。

8. 纳入材料说明

- 已纳入：前端核心源码、辅助服务源码、工程配置、入口页面、演示场景配置。
- 未纳入：第三方依赖、构建产物、静态资源文件、无关缓存目录。
- 特别说明：`src/js` 目录为历史独立脚本版本，当前单页应用未直接引用，但属于项目真实原创代码，故纳入材料作为参考与演进说明。